

Samuli Reinikainen

GENETIC ALGORITHM FOR PRUNING TRANSFORMERS

Master of Science Thesis
Faculty of Information Technology and Communication Sciences (ITC)
Examiners: Jaakko Peltonen
Martti Juhola
June 2024

ABSTRACT

Samuli Reinikainen: Genetic Algorithm for Pruning Transformers
Master of Science Thesis
Tampere University
Data Science
June 2024

This thesis introduces a genetic algorithm based pruning approach for Transformer neural networks. Large Transformer neural networks, consisting of billions of parameters, have achieved state-of-the-art results in many machine learning tasks, especially in the domain of natural language processing. The size of these models and of deep learning models in general, sets high computational demands for inference which can form a significant hurdle for wider adoption. Pruning provides a way of reducing the parameter count of the model, making the model more sparse and inference less demanding. The goal of this thesis is to explore a genetic algorithm based method for pruning and determine whether it is a viable method for pruning Transformers.

The genetic algorithm based pruning framework consists of a genetic algorithm, which searches for optimal parameters to remove from a neural network optimized for a specific learning task. Mask variables are constructed for each parameter matrix of the model which are then flattened to form a binary chromosome. A population of such chromosomes is developed by the genetic algorithm to find an optimal chromosome, which is then transformed back to mask variables. The removal of parameters by the genetic algorithm and further training of the model are iterated until a specified level of sparsity is reached.

An experimental case study on BERT, a Transformer-based language model, is presented to test the approach for two widely used text classification benchmark datasets. The proposed approach achieved a significant sparsity level with a small decrease in accuracy, outperforming a baseline pruning method by a large margin. The results are comparable to state-of-the-art pruning methods for Transformers. Despite the promising results, more thorough and diverse computational experiments are required to assess the performance of the approach completely.

Keywords: machine learning, Transformer, neural network, pruning, genetic algorithm, natural language processing

The originality of this thesis has been checked using the Turnitin OriginalityCheck service.

TIIVISTELMÄ

Samuli Reinikainen: Geneettinen algoritmi Transformer-neuroverkkojen karsintaan
Pro gradu -tutkielma
Tampereen yliopisto
Datatiede
Kesäkuu 2024

Tämä tutkielma esittelee geneettiseen algoritmiin perustuvan karsintamenetelmän Transformer-neuroverkoille. Suuret Transformer-pohjaiset neuroverkot, jotka koostuvat usein miljardeista parametreista, ovat saavuttaneet huipputuloksia monissa koneoppimisen tehtävissä, erityisesti luonnollisen kielen käsittelyn alueella. Näiden mallien – ja yleisesti ottaen syväoppimismallien – koko asettaa korkeat laskennalliset vaatimukset niiden hyödyntämiselle, mikä voi muodostaa merkittävän esteen mallien laajemmalle käyttöönotolle. Karsinta on tapa vähentää mallin parametrien määrää, mikä tekee mallista harvemman ja päättelystä vähemmän laskennallisesti vaativaa. Tämän tutkielman tavoitteena on tutkia geneettiseen algoritmiin perustuvaa karsintamenetelmää ja selvittää, onko se toteuttamiskelpoinen menetelmä Transformer-neuroverkkojen karsintaan.

Esitelty geneettiseen algoritmiin perustuva karsintamenetelmä koostuu geneettisestä algoritmista, joka etsii optimaalisia parametreja poistettavaksi tiettyyn oppimistehtävään optimoidusta neuroverkosta. Jokaiselle mallin parametrimatriisille muodostetaan maskimuuttuja, jotka yhdistämällä muodostetaan binäärinen kromosomi. Geneettinen algoritmi kehittää tällaisista kromosomeista koostuvaa populaatiota, löytääkseen optimaalisen kromosomin, joka muunnetaan lopuksi takaisin maskimuuttujiksi. Parametrien poistamista geneettisen algoritmin avulla ja mallin jatkokouluttamista toistetaan, kunnes tavoiteltu harvuustaso saavutetaan.

Tutkielmassa esitetään kokeellinen tutkimus, joka hyödyntää Transformer-pohjaista BERT-kielimallia, jonka avulla esitettyä karsintamenetelmää testataan kahdella laajalti käytetyllä tekstiluokitteluaikoneistolla. Esitetty menetelmä saavutti merkittävän harvuustason pienellä tarkkuuden laskulla, suoriutuen huomattavasti paremmin kuin vertailussa käytetty peruskarsintamenetelmä. Saadut tulokset ovat vertailukelpoisia parhaimpiin kirjallisuudessa esitettyihin Transformer-karsintamenetelmiin. Lupaavista tuloksista huolimatta menetelmän suorituskyvyn tarkkaan arviointiin tarvitaan kattavampia ja monipuolisempia laskennallisia kokeita.

Avainsanat: koneoppiminen, Transformer, neuroverkko, karsinta, geneettinen algoritmi, luonnollisen kielen käsittely

Tämän julkaisun alkuperäisyys on tarkastettu Turnitin OriginalityCheck -ohjelmalla.

CONTENTS

1. Introduction	1
2. Artificial Neural Networks	4
2.1 Structures of Artificial Neural Networks	5
2.2 Activation Functions	7
2.3 Tasks and Learning	8
2.4 Training Neural Networks	9
2.5 Overfitting and Regularization	11
2.6 Hyperparameters	12
2.7 Transfer Learning	13
2.8 Neural Network Architectures	14
2.9 Deep Learning	15
2.10 Natural Language Processing	16
3. Transformer Neural Networks	18
3.1 Attention Mechanism	18
3.1.1 General Attention Mechanism	18
3.1.2 Attention in Neural Networks	19
3.2 Transformer Architecture	21
3.2.1 Attention in Transformers	22
3.3 Bidirectional Encoder Representations from Transformers	23
3.3.1 Architecture	24
3.3.2 Pretraining and Finetuning	25
4. Neural Network Pruning	27
4.1 Theoretical Framework of Pruning	28
4.1.1 Computations of a Pruned Neural Network	29
4.2 Pruning Neural Network Structures	29
4.3 Pruning Criteria	31
4.4 Pruning Strategies	32
4.5 Pruning Transformers	32
5. Genetic Algorithms	34
5.1 Search Problems	34
5.2 Structures of Genetic Algorithms	35
5.2.1 Gene Encoding	35
5.2.2 Crossover	36
5.2.3 Selection	38

5.2.4	Fitness Evaluation	39
5.2.5	Mutation	39
6.	Methodologies.	41
6.1	Pruning with Genetic Algorithms.	41
6.1.1	Gene Encoding for Pruning	41
6.1.2	Fitness for Pruning	43
6.2	Proposed Framework	43
6.3	Experiment Details	45
6.3.1	IMDB and AG News Datasets	46
6.3.2	Training and Pruning Details.	47
6.3.3	Algorithm Analysis	48
7.	Results	50
7.1	Overall Results	50
7.2	AG News	51
7.3	IMDB	55
7.4	Discussion	58
7.4.1	Comparison to Previous Results on Transformer Pruning	59
7.4.2	Limitations and Further Research Directions	60
8.	Conclusion	61
	References.	62

1. INTRODUCTION

Deep learning methods have achieved state-of-the-art results on many machine learning tasks in the last 20 years [1]. During this time, many innovations have improved previous deep learning approaches in different problem domains by large leaps in performance. Recently, one of the most important of such innovations has been the development of the Transformer neural network architecture, which has enabled revolutionary results in many deep learning applications on sequential data, most notably natural language processing [2][3]. Large pretrained Transformer-based language models such as Bidirectional Encoder Representations from Transformers (BERT) and the Generative Pretrained Transformer (GPT) series have seen nearly unmatched interest in the Artificial Intelligence (AI) community as well as in the general public [3]. This innovation was among the most important driving factors behind the ongoing rapid development of generative AI. Large Language Models (LLMs), such as GPT-4, have been heralded as a leading example of what could be considered an early stage of Artificial General Intelligence (AGI), which is regarded as a pivotal goal within the field of AI research [4].

One significant barrier to the broader adoption of deep learning, including Transformer neural networks, is the large amount of computing power and memory required to utilize them efficiently. The sheer size of the state-of-the-art models sets strict requirements for inference hardware and software and even with modern technology, the latency of inference can become a bottleneck for many tasks. This is especially true for large pretrained Transformer neural networks which can consist of hundreds of billions of parameters [3]. However, in the case of large pretrained models, it has been shown that a significant amount of the parameters are not being utilized for many specific applications [5].

Many methods have been developed for managing the computational demand caused by the large number of parameters [6]. Methods that try to reduce the computational cost by manipulating the architecture and parameters of the network include pruning, quantization and knowledge distillation. These methods, known as model compression techniques, attempt in different ways to limit the complexity of the neural network in order to remove redundant or unimportant information from its parameters. Model compression typically results in a lower cost of inference and reduced inference latency, with the trade-off of some lost performance [5].

Pruning is a model compression technique where individual parameters or groups of parameters are deleted from the neural network in order to reduce its size. Finding an optimal way of removing parameters can be formulated into an optimization problem where the parameter space is searched for an ideal subset of parameters to remove. Many heuristic solutions for this optimization problem have been proposed. The most common heuristics use norms of the parameters or gradient information gathered during training as proxies for the importance of a parameter [7][8].

Genetic algorithms are a class of heuristic search algorithms inspired by genetics and evolution. Genetic algorithms are well suited for complex optimization problems, as they can provide a good balance between exploration of the space of possible solutions and exploitation of previously found solutions. Although genetic algorithms have seen wide use in neural architecture search [9], there has been limited research on utilizing genetic algorithms for pruning neural networks, especially applied to Transformers.

The goal of this thesis is to experiment with a genetic algorithm based pruning approach for Transformers and provide a new research direction in Transformer compression. Further, this thesis aims to answer the question, whether a genetic algorithm based pruning approach is a feasible option for large deep learning models. To answer the question, a case study of the pruning approach is presented on BERT, a pretrained Transformer-based language model.

The goal of this thesis is to experiment with a genetic algorithm-based pruning approach for Transformers. The research questions of the thesis are, whether a genetic algorithm-based pruning approach is a feasible option for large deep learning models and how this approach can provide a new research direction in Transformer compression. To answer these questions, the thesis presents a case study of the pruning approach on BERT, a pretrained Transformer-based language model.

In the first four chapters, the background theory of the main components of the thesis, Transformers neural networks, pruning and genetic algorithms, is explored. In Chapter 2, the basic theory related to artificial neural networks and their use in machine learning is presented. The various components of neural networks and their training processes are highlighted in order to provide a conceptual basis for presenting the genetic algorithm based pruning approach. In Chapter 3, the Transformer architecture and the attention mechanism central to it are examined. BERT, a language model based on the Transformer architecture that is used in the case study, is also explored. In Chapter 4, the theoretical framework of pruning neural networks is presented and different approaches to neural network pruning are highlighted. The specifics of pruning Transformer-based models, especially BERT are also discussed. In Chapter 5, the theory of genetic algorithms and their use in solving search problems are explored. The chapter highlights different components of genetic algorithms and how they can be used to solve a wide

range of optimization and search problems.

In Chapter 6 the specific methodologies used in the implementation of the genetic algorithm based pruning approach are examined. The details of the case study demonstrating the feasibility of the method is also presented. In chapter 7, the results of the experiments in the case study are presented and the limitations of the method and possible further research directions for genetic algorithm based pruning are discussed.

2. ARTIFICIAL NEURAL NETWORKS

In this chapter, general background theory of machine learning and artificial neural networks is presented. The goal of this chapter is to introduce the general concepts and theory behind neural networks in a level that is required for discussing the Transformer neural network architecture and neural network pruning in later chapters. First, the basic structures of artificial neural networks and their roles in the computations are presented. Next, the training procedures of neural networks as well the topic of overfitting and regularization are discussed. Different neural network architectures are introduced briefly with an emphasis on those relevant to the Transformer architecture.

Artificial neural networks (ANNs) are a class of machine learning algorithms loosely inspired by the structure of the human brain. These networks consist of interconnected computational units referred to as artificial neurons, or nodes, which perform sequential computations on input data until an output is reached. The nodes of the ANN are connected with weighted connections which are used in computing the value of each node in a sequential manner, until an output is reached. These weights, or parameters, encode all of the information that the model learns when trained for a specific task.

As is the case for any machine learning system, the goal of artificial neural networks is to learn to perform a task by building a model based on some training data. Historically, various different types of artificial neural networks have been utilized for different learning tasks. One of the primary learning paradigms utilized within the domain of ANNs today is supervised learning. There, an ANN acts as a function approximator, attempting to learn an approximate relationship between the input features X and an output variable y . The neural network learns to perform the function approximation task by optimizing its parameters so that a predefined cost function is minimized. The cost function typically quantifies the difference between the network's predictions and the training data, thus serving as a measure of the ANN's performance in the learning task. By adjusting the network's parameters iteratively, the ANN improves its ability to predict the values of the output variable accurately. This process is continued until the approximate mapping between the inputs and outputs is a good enough approximation to perform the task on new unseen data.

2.1 Structures of Artificial Neural Networks

Neural networks come in a variety of different structural formats, known as neural network architectures. The most typical neural network architecture, called a feed forward neural network, consists of layers of connected artificial neurons that allow the information to flow in one direction only, forming a directed acyclic graph [10]. The graph is directed, because the connections are one-way and acyclic as information flows through the network without cycling back to visited neurons. Each neuron performs a weighted sum on its input data and passes the output on to one or multiple other neurons. The calculation performed by a single artificial neuron is

$$g(\mathbf{x}) = \mathbf{w}^T \mathbf{x}, \quad (2.1)$$

where $\mathbf{w} \in \mathbb{R}^m$ is a vector of the parameters of connections coming into the neuron and $\mathbf{x} \in \mathbb{R}^m$ is a vector of real-valued input values [11]. An optional bias term $b \in \mathbb{R}$ is often included into the sum in equation 2.1. The input values x can be outputs of preceding neurons or the original input features depending on the position of the neuron in the network. If an output is passed from one neuron to another, the neurons are said to be connected. The output $g(x)$ of a neuron is also known as its activation, a term of biological origin. Typically, a nonlinear activation function ϕ is applied to the output of a neuron before passing it as input to other neurons. The specific role of activation functions is described later in this chapter.

The neurons of a feed forward neural network can be organised into groups known as layers, such that the neurons in the preceding layer are connected to the neurons in the next layer. The computation of a single layer can then be described as a single matrix multiplication by combining all of the weights between the layers into a weight matrix [11]. Specifically, by stacking the parameter vectors of all connections coming into the k neurons of a layer into a parameter matrix $W \in \mathbb{R}^{m \times k}$, the calculations of a layer can be formulated as

$$g(x) = W^T \mathbf{x}. \quad (2.2)$$

Each layer of such a network thus calculates a linear mapping $\mathbb{R}^m \rightarrow \mathbb{R}^k$ and a nonlinear activation function and feeds the result forward to the next layer.

A feed forward neural network is composed of one or multiple of the described computational layers, nonlinear activation functions between the layers, as well as an input layer. The input layer consists of placeholder nodes that bring in feature values to the network but perform no actual calculations. The computational layer that performs the final calculations is known as the output layer and its activations form the final output of the neural network. Layers between the input and output layers are referred to as hidden layers. The amount of layers in the neural network and the amount of neurons in a single layer are design choices made by the experimenter. Although an ANN with more layers and

nodes can in theory represent more complex functions, the optimal choice of layer and node count is highly dependent on the amount of training data and the complexity of the learning problem. These design choices are examples of hyperparameters, which are described in more detail later in this chapter.

A neural network, which has connections between all nodes of consecutive layers is known as a fully connected neural network. Figure 2.1 contains a diagram depicting a graph representation of a fully connected feed forward neural network with eight input features, one output and two hidden layers with five and four neurons respectively. The nodes of the graph represent the neurons of the ANN and the edges represent a connection between two nodes. The weights of the connections between the layers are omitted from the diagram.

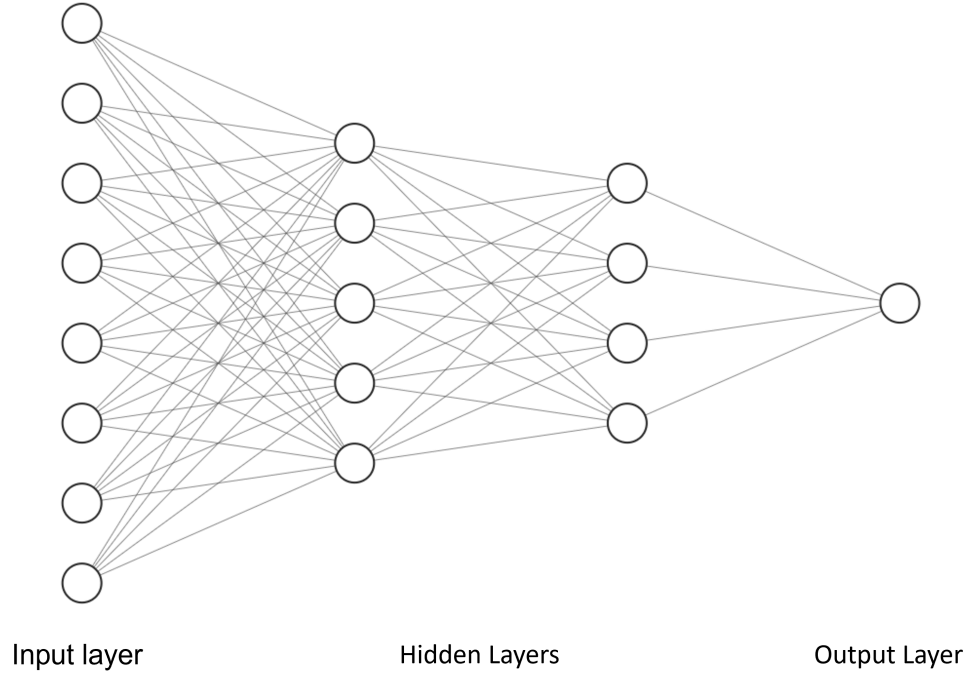


Figure 2.1. A diagram depicting a graph representation of a fully connected feed forward neural network

In total, the calculations from input features to the final output of a feed forward neural network, can be represented as a composite function of multiple linear transformations and nonlinear activation functions applied elementwise. This can be denoted as

$$\hat{f}(x) = (\phi_L \circ g_L \circ \phi_{L-1} \circ g_{L-1} \circ \dots \circ \phi_1 \circ g_1)(x) \quad (2.3)$$

for a network with L computational layers. One evaluation of this composite function for a specific value of x is known as a forward pass.

2.2 Activation Functions

Activation functions are nonlinear mappings $\mathbb{R} \rightarrow \mathbb{R}$ applied to the activations of a node before passing as input to other nodes. They serve mainly two purposes within the network architecture: providing non-linearity to the computations and constraining the range of node output values. Most activation functions can be applied to the outputs of individual nodes but some require the activations of the entire layer to calculate the outputs, most notably the softmax function described later.

The most important role of activation functions is providing non-linearity to the computations of the neural network. Applying only linear operations within the layers constraints the entire network to modeling linear relationships between the input features and the target, regardless of the number of layers in the network [10]. Non-linear activation functions are thus crucial for enabling neural networks to approximate non-linear functions.

Activation functions are also used to limit the range of intermediate values in the computational layers of the network, to avoid extreme values causing computational or memory issues when training the network. The activation functions of the output layer are designed to transform the output of the neural network to a desired range or to make them compatible with a specific output datatype. [10]. A typical example is transforming activations to the range $[0, 1]$ when the desired output is a vector of probabilities of some event, typically class membership in a classification problem.

Inspired by synapses of the human brain, the first ANNs used a simple step function as the activation function [12]. Later, the development of gradient based training algorithms for neural networks lead to the need of having differentiable activation functions. Thus, different types of sigmoid, or S-shaped, activation functions such as the logistic function and the hyperbolic tangent function gained popularity [10]. Sigmoid functions approach a constant value as the input approaches positive or negative infinity, essentially providing a continuous, differentiable alternative to the step function. The logistic sigmoid activation function is defined as

$$\phi(x) = \frac{e^x}{e^x + 1}. \quad (2.4)$$

The logistic sigmoid function has an output range of $[0, 1]$ and is still often used in the output layers for binary classification tasks to represent class probabilities.

Later, it was discovered that for ANN's with a high number of layers, so called rectifier activation functions have better theoretical and practical properties for intermediate computational layers [10]. Today, many activation functions used in the hidden layers of neural networks are variations of the Rectified Linear Unit (ReLU). The ReLU activation function is defined as

$$\text{ReLU}(x) = \begin{cases} x & \text{if } x \geq 0 \\ 0 & \text{if } x < 0. \end{cases} \quad (2.5)$$

Besides its computational simplicity, the advantage of Rectified Linear Unit over sigmoid activation functions has to do with the gradient of the function, used in most training algorithms. For sigmoid functions, the derivative approaches zero for both sufficiently positive and negative values. This can lead to the so-called vanishing gradient problem, which is somewhat alleviated by the ReLU, having a gradient of either one or zero.

Various modifications of the ReLU activation function have been proposed, such as LeakyReLU [13] and most recently, the Gaussian Error Linear Unit (GELU). The GELU activation function has seen wide usage in deep neural networks and especially Transformer-architectures such as BERT, discussed later in this chapter. For the definition of GELU, let Z be a standard normal random variable $Z \sim N(0, 1)$. The GELU activation function is defined as

$$\text{GELU}(x) = xP(Z \leq x). \quad (2.6)$$

GELU functions essentially as a smoother version of ReLU [14].

Another activation function relevant to this thesis is the softmax function. Softmax transforms a vector of numbers into a probability distribution according to their relative sizes. It is essentially a multidimensional version of the logistic function. The softmax function is often used in the last layer of a multiclass classification neural network in order to transform the activations of the output layer to a class probability distribution [10]. For the softmax function, the calculation of a single node's activation is dependent on all the activations of the layer. Let the vector of activations be $\mathbf{a} = (a_1, a_2, \dots, a_k)'$ for an output layer with k neurons. The softmax activation function is defined as

$$\text{softmax}(\mathbf{a}) = \begin{bmatrix} \frac{e^{a_1}}{\sum_{i=1}^k e^{a_i}} \\ \vdots \\ \frac{e^{a_k}}{\sum_{i=1}^k e^{a_i}} \end{bmatrix}. \quad (2.7)$$

2.3 Tasks and Learning

In machine learning, a task T refers to a specific problem or objective that is to be solved or accomplished by the machine learning algorithm. Examples of machine learning tasks include classification, regression, anomaly detection and machine translation. To learn to do a task, the algorithm experiences a set of examples. Examples are the units which the machine learning algorithm is expected to process to complete the task [10]. An individual example is represented by a collection of feature values which typically form a real-valued vector $x \in \mathbb{R}^p$. Typically, learning a task requires a set of multiple examples $X = \{x_i\}, i = 1, 2, \dots, N$ and the corresponding desired outputs $y_i, i = 1, 2, \dots, N$. The combination of the two is often called the training data set. Finally, a quantitative performance measure P is needed to evaluate the performance of the algorithm on the task.

Training a neural networks is the process of optimizing the parameters of the network to solve a specific learning task T given the performance measure P and the training dataset X .

Machine learning tasks can be informally divided into two categories based on the type of training dataset the algorithm is given access to. In unsupervised learning, the learning algorithm experiences a set of training examples and attempts to learn some aspect of the probability distribution $p(X)$ which generated the data. One example is clustering, where the task is to assign the training examples into clusters such that similar training examples end up in the same cluster.

As mentioned, one of the most important learning paradigms for neural networks is supervised learning. In supervised learning the algorithm experiences a set of training examples and a set of labels $Y = \{y_i\}, i = 1, 2, \dots, N$, one for each example. The goal is then to learn a relationship between x and y , typically an approximate mapping $x \rightarrow y$. Supervised learning tasks can further be divided by the type of inputs and outputs of the mapping. In classification, the goal is to assign one or multiple class labels to a data item according to the input features. In regression, the goal is to predict a continuous output variable. Both supervised learning tasks can also be extended to include multidimensional output vectors and sequences. A specific type of supervised learning task relevant to this thesis is the sequence-to-sequence model. There, both the example and the label are sequences of real values or class labels.

Another way of utilizing unlabeled training examples is self-supervised learning. In self-supervised learning, the task is to learn a mapping between a specific characteristic of the input and the input features. In practice, this is often achieved by creating so-called pseudo-labels for the data items by masking a part of the input during training. The features that are left unmasked are used to predict the masked features. An example of this is in masked language modelling utilized in BERT, discussed in Chapter 3, where the goal is to learn a mapping between words in natural language text.

2.4 Training Neural Networks

A neural network model is a function composed of multiple simpler functions which depend on parameters $\theta_i \in \mathbb{R}$. By denoting the set of all parameters by Θ , we can represent the neural network by a function $f(x; \Theta)$. The training of neural networks is the process of optimizing the parameter values Θ of the network f to solve a learning task as well as possible according to a performance measure $P : \Theta \rightarrow \mathbb{R}$ over a probability distribution of data $p(x)$. Including the labels y for supervised learning, the optimization is done over $p(x, y)$. In many cases, P is not differentiable and can thus not be optimized directly, but instead a differentiable surrogate cost function $c : \Theta \rightarrow \mathbb{R}$ is optimized instead. For supervised learning, the optimization problem can be formalized as

$$\underset{\Theta}{\text{minimize}} \quad c(\Theta, p(x, y)). \quad (2.8)$$

It is typically assumed that the cost function $c(\Theta)$ can be seen as an expected value of another function applied to individual data examples in $p(x, y)$, known as a loss function and denoted as $\mathcal{L}(f(x; \Theta), y)$. The cost function can then be represented as

$$c(\Theta) = \mathbb{E}_{X,Y}(\mathcal{L}(f(x), y)). \quad (2.9)$$

This quantity is also called the theoretical risk [10].

In practical use cases however, one does not have access to the entire data generating distribution $p(x, y)$ but instead a set of N training examples generated from $p(x, y)$ denoted as (x_i, y_i) for $i \in \{1, \dots, N\}$. The simplest way to form the optimization then is then to estimate the cost function using an average quantity known as the empirical risk, defined as [10]

$$\frac{1}{N} \sum_{i=1}^N \mathcal{L}(\hat{f}(x_i), y_i). \quad (2.10)$$

Minimizing the empirical risk over the parameters is an unconstrained optimization problem. As the cost functions of neural networks generally turn out to be non-convex, gradient based optimization algorithms are commonly utilized for solving them. These methods utilize the gradients of the loss function with respect to the parameters Θ and incrementally adjust the parameters to the direction opposite to the gradient in order to arrive at a sufficiently small value of the empirical risk function. A general rule for updating an individual parameter θ_i in gradient descent optimization is

$$\theta_i = \theta_i - \alpha \frac{\partial \mathcal{L}}{\partial \theta_i}, \quad (2.11)$$

where α is the learning rate, a number that controls the speed of convergence of the algorithm. Calculation of the gradients of the loss function involves using the chain rule of differentiation on the function in (2.3). An efficient algorithm for calculating the gradients is known as backpropagation [15]. The name is also used when referring to gradient descent algorithms that utilize it.

Computing the gradients of the loss function over the entire training data set for each parameter update can become computationally expensive. One can however estimate the gradients by taking a random sample of n training examples and calculating the gradients using them. This approach is known as minibatch gradient descent and the number n is known as the batch size. When $n = 1$, the name stochastic gradient descent is also

used [10]. The resulting training algorithm is iterated until a specific stopping criterion is achieved. Often this requires updating the parameter values multiple time for each training example. The training process is typically divided into so-called epochs, where a single epoch consists of calculating the gradients and updating the parameter values once for each training example.

As stated before, the cost functions of neural networks turn out to often be non-convex, meaning that they have local minima that optimization algorithms can get stuck to. Various alterations to the basic gradient descent algorithm have been developed to tackle the problems of local minima and to reduce the number of iterations required for converging to a minimum. Adding a so-called momentum term calculated from moving averages of the gradients of past iterations, to the updating equation in (2.11) has been shown to accelerate convergence in many cases and also help with local minima. Some algorithms, such as the adaptive gradients optimizer (AdaGrad) [16], and its extensions AdaDelta [17] adjust the learning rate of individual parameters to accelerate learning. This has been shown to be especially useful when dealing with sparse input features. To achieve this, the algorithm stores a list of previous gradient values of each parameter and scales the learning rate accordingly. Adaptive moments estimation algorithm (Adam) further improves the adjustments done to the learning rate by utilizing the first and second moment of the gradient.

2.5 Overfitting and Regularization

Employing the empirical risk estimate of the cost function, as shown in Equation 2.10 comes with the assumption that reducing the empirical risk will concurrently decrease the actual cost function. This is often not the case though leading to a phenomenon known as overfitting or overlearning. Overlearning happens when a model achieves high performance on the training data set but generalizes poorly to new data coming from $p(x, y)$. In overfitting, the model optimizes its parameters to random variations specific to the used training data set which are not a feature of the data generating distribution in general. The training performance may increase indefinitely but the performance on data coming from the theoretical $p(x, y)$, known as the generalization performance, may stagnate or even start decreasing.

Regularization refers to techniques that attempt to mitigate overfitting. In general, a regularization method is any method that aims to improve the generalization performance of a machine learning algorithm without considering the performance on the training set. Some of the most notable techniques proposed include constraining parameter norms, employing priors for parameters, parameter sharing, training dataset augmentation, dropout and early stopping [10].

Constraining parameter norms refers to augmenting the cost function with a penalty term

that penalizes large values of parameters. The regularized cost function becomes

$$c'(\Theta|X, y) = c(\Theta|X, y) - \Omega(\Theta), \quad (2.12)$$

where $\Omega(\Theta)$ is the parameter norm penalty. L_1 and L_2 norms are popular choices for Ω . Constraining parameters forces the model to search for solutions in a much more narrow space of parameters thus reducing its complexity. This in turn may reduce the chance of fitting to random variations of the dataset. [10]

Dropout is a regularization technique where a fraction of the neurons of a neural network are shut off randomly during an iteration of training. In other words, for each iteration of the training algorithm a node is kept in the network with probability p . The dropout probability is a number chosen by the experimenter. As a result of the dropout operation, a slightly different network architecture is trained on each iteration of the training process. The resulting model can be seen as an averaged model of the different architectures used during training. Dropout regularization has been shown to increase the generalization performance in various tasks [18].

Early stopping refers to the technique of stopping training before the empirical risk function has converged. The goal of early stopping is to stop training when the model has learnt the general features of the data generating distribution but has not yet optimized its parameter values to random variations specific to the training dataset in question. This leads to a lower training performance but can lead to an increased generalization performance [10]. The most common way to implement early stopping is by introducing a hold-out validation set into the training algorithm. The hold out validation set is a separate set of examples sampled from the data generating distribution independent of the training examples. The value of the cost function over the hold-out validation set is evaluated periodically during training, usually once per epoch, and the training process is terminated if the value stops decreasing for an extended amount of iterations.

2.6 Hyperparameters

Choices made by the experiment designer that affect the structure or training of a neural network but are not optimized in the training process are called hyperparameters [10]. Examples of hyperparameters include the number of layers, the number of artificial neurons within each layer and the activation functions used. Hyperparameters that affect training include the batch size and the learning rate as well as hyperparameters that control the momentum and learning rate adjustments of various optimization algorithms. Regularization techniques can also have hyperparameters such as the dropout probability in dropout regularization. Choosing hyperparameters that alter the architecture of the neural network is generally known as neural architecture search [9].

The choice of hyperparameters is generally made heuristically, although different types of hyperparameter search algorithms have been proposed. These often include training the neural network multiple times with different hyperparameters, measuring the generalization performance using a validation dataset and choosing the best performing one. Hyperparameters can also be altered during training, an example of which is the altering of the learning rate by various optimization algorithms. Iterative pruning during training, which can be seen as a type of neural architecture hyperparameter search is discussed in later in Chapter 4. Further, the genetic algorithm based pruning method presented in Chapter 6 can also be seen as a type of neural architecture search.

2.7 Transfer Learning

Transfer learning refers to applying knowledge learnt from one learning task to solve another task. There are many ways to implement transfer learning with neural networks but typically they include sharing parameters of the neural networks between tasks. For two tasks with similar inputs, an architecture where the tasks share the first layers of the network but have different output layers is a common choice. Another common way of implementing transfer learning is to train the neural network for one task with a large training dataset and then utilizing the parameters of the trained network for another similar task that has less training data. The words pre-training and finetuning are often used in this context. The aim of pre-training is to improve generalization performance on the task with less data and to speed up the convergence of training. Thus it can also be seen as a type of regularization.

There are many ways of utilizing a pretrained network on other tasks. The entire pre-trained model architecture with all its parameters can be further trained on the finetuning task if the tasks are identical up to the specific training datasets used. If the learning tasks are substantially different but utilize similar data, it can be reasonable to use only a specific part of the pretrained architecture and connect them to further layers in the finetuned network architecture. This allows for flexibly utilizing the useful representations that the original neural network learned during pretraining.

It is also possible to freeze parts of the pretrained network so that their parameters will not be altered by gradient descent during finetuning. If the entire utilized section of the pre-trained architecture is frozen during finetuning, the word feature extraction is often used. There, the pretrained architecture merely transforms the input features into more useful representations that are then fed as input to a completely separate finetuned model.

2.8 Neural Network Architectures

The types and amounts of layers and neurons used as well as the connections between them define the neural network architecture. Many types of variations on the basic feed forward neural network have been proposed. A variety of specialised computational units and ways of connecting them have been developed to suit the needs of different types of learning tasks. By designing a specific neural network architecture, the experimenter applies an inductive bias to the learning of the model. A well designed inductive bias can help the model learn more complex tasks with a smaller network and less data.

Recurrent neural networks (RNN) have connections that can form cycles, where a neuron passes data onto itself or onto other neurons between different iterations. RNNs are mostly used in the context of sequential data, in which the data items have an inherent ordering. The activations from the computations of the previous step in the sequence, called a hidden state, are passed onto the same neuron as inputs to the computation of the next step. Recurrent layers have been found to be highly useful in sequence-to-sequence modeling, where the goal is to learn a mapping between sequences of possibly different length. Recurrent neural networks are designed to have an inductive bias to consider the ordering of the sequence. Although a basic feed forward network could in principle learn the same mapping, the inductive bias can make the learning process more robust for the recurrent network. Often, recurrent neural networks are used bidirectionally, by concatenating the outputs of the recurrent layers after computing the forward pass starting from both ends of the sequence.

Encoder-decoder neural network architectures are another architecture typically seen in sequence-to-sequence modelling and especially in natural language processing tasks such as machine translation. In principle, encoder-decoder networks can be used for any other type of data. An encoder-decoder network consists of two more or less separate neural networks, an encoder and a decoder. Figure 2.2 presents an illustration of an encoder-decoder architecture. The encoder networks takes an input sequence and encodes it into a compressed latent representation. The compressed representation is then fed to the decoder-network which can decode a variable length output sequence from the representation [19]. These two networks can be trained together for a supervised sequence-to-sequence task. If the task is to reconstruct the original input sequence from the intermediate representation, the term autoencoder is also used. Recurrent neural networks have been traditionally utilized as the encoder and decoder in encoder-decoder models used for machine translation. These types of models later paved the foundation for Transformer neural networks, which are themselves types of encoder-decoder neural networks, discussed in Chapter 3.

Many other types of layers and neural network architectures have been developed for handling different types of input data and learning tasks as well as making training neu-

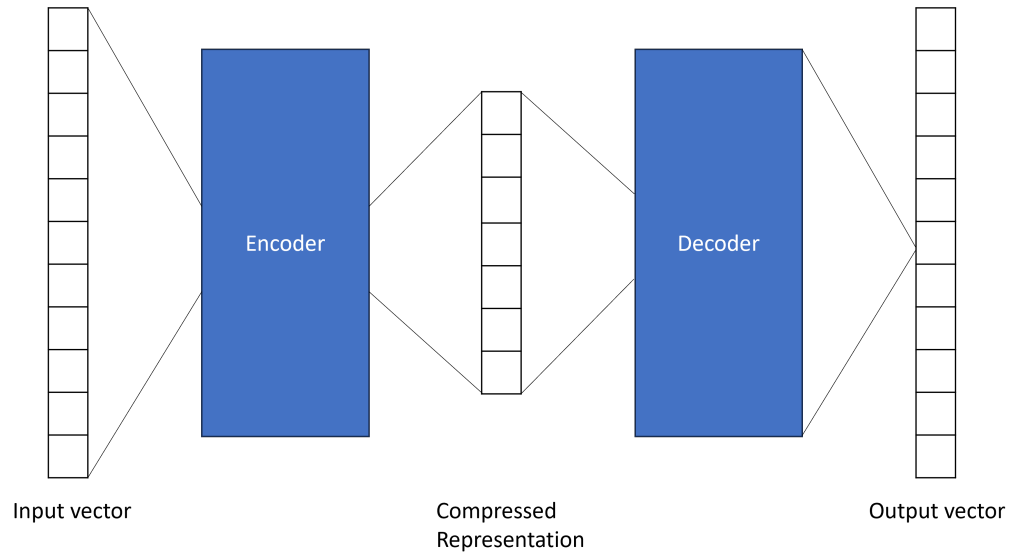


Figure 2.2. An illustration of an encoder-decoder neural network architecture

ral networks more efficient. Convolutional neural networks (CNN) utilize a mathematical operation called convolution instead of a simple matrix multiplication in some of their layers. They have been designed to process so-called grid data types, including time series, images and video. Instead of having a parameter for each position in the grid, a convolutional layer uses grids of parameters called convolution kernels that are slid over the input grid. This is a form of parameter sharing which can make training more robust and increase generalization performance [10]. Residual neural networks include skip connections and gating mechanisms that enable jumping over layers in specific situations which can help optimization algorithms converge [20]. Long term short term memory (LSTM) neural networks, a type of RNN, include gating mechanisms which make them more robust to train [21]. Recently, so-called graph neural networks (GNN) have been developed to process data that can be represented as a graph [22].

2.9 Deep Learning

Although in theory, a neural network architecture with one hidden layer and a nonlinear activation function has been shown to be able to model virtually any real valued function to arbitrary accuracy given enough nodes, it is often more practical to use an architecture with more layers and less nodes. [23] It has been shown that in many cases, a network with more layers is able to represent most functions with a smaller total number of parameters than a shallower network. Network architectures with more layers have been empirically shown to lead to an empirical risk which better approximates the theoretical risk and thus results in a model that generalizes better. [10] These discoveries led to the popularization of neural networks with more and more layers and eventually the term deep learning was coined. The depth refers to the large amount of layers of the neural

network.

A downside to creating increasing the size and scale of neural networks is that the amount of parameters can quickly become large. In fact, many popular deep neural network architectures are heavily overparameterized, meaning that they have more parameters than there are training examples used to train them. It has been shown that deep learning architectures end up not using a large portion of their parameters in a specific learning task. This phenomenon is especially prevalent with neural networks utilizing transfer learning (see for example [5], [24] and [25]). One way to combat the problems caused by the size of the models is neural network pruning, which refers to retrospectively deleting redundant connections or nodes from a trained network in order to reduce computational costs and increase generalization performance. Pruning will be covered in detail later in Chapter 4.

2.10 Natural Language Processing

Neural networks have seen wide usage in various natural language processing tasks for decades. Using neural networks for tasks such as text classification and semantic search have been around for years, and lately the Transformer-based GPT-series contributed to the popularity of text generation tasks. Utilizing text as an input for neural networks requires it to first be encoded into a vector of real numbers. Many different methods for vectorizing text have been proposed [26]. Some methods create a single vector for a text comprising multiple words, whereas other methods create vectors for individual words or other lexical units. A trivial way of creating vectors out of text, is to create an ordered set of all possible words in the corpus, known as a dictionary $D = (w_1, w_2, \dots, w_n)$. Then each word w_k is described by a binary vector which has a 1 in index j and is zero every where else. More formally w_k is represented by

$$[i_1, i_2, \dots, i_n], \quad i_j = \begin{cases} 1 & \text{if } w_j = w_k \\ 0 & \text{otherwise.} \end{cases} \quad (2.13)$$

A text consisting of multiple words would then be represented by an ordered set, or sequence of such binary word vectors. The approach can also be directly applied to other lexical units than words. Many modern applications use word piece embeddings, where a single word can be divided into shorter combinations of characters, often referred to as tokens.

The problem with directly utilizing such vectors is that they can become very large in dimensionality. Dictionaries can consist of hundreds of thousands of words, which is directly reflected in the vector dimensionality. Embedding layers are specialized neural network layers or architectures, which transform such sparse input vectors into a dense

representation space with lower dimensionality, typically if a few hundred dimensions. Such dense representation vectors are called word embeddings. Word embeddings are typically trained in such a way that semantically similar words yield embeddings that are close in proximity to each other in the embedding vector space, enabling their use in applications such as semantic search. Word embedding layers are typically used as the first computational layers of language models. An example of this is the BERT model [27], described in Chapter 3.

3. TRANSFORMER NEURAL NETWORKS

In this chapter, details of the Transformer neural network architecture are presented. The Transformer neural network is an encoder-decoder sequence-to-sequence neural network that utilizes the attention mechanism which allows it to pay attention to relevant positions in the input sequence when decoding the output sequence. First, the building block of Transformer neural networks, the attention mechanism is investigated. Next, the Transformer architecture and the utilization of attention mechanisms within it are discussed. Finally, the architecture and use cases of Bidirectional Encoder Representations from Transformers (BERT), a large Transformer-based pretrained language model that is used in the case study in chapter 7 is examined.

3.1 Attention Mechanism

At the core of most state-of-the-art models in domains like natural language processing is a so-called attention mechanism. The term originates from cognitive attention which refers to the capability of concentrating on a specific, discrete part of information while dismissing other available information. The attention mechanism of neural networks is based on applying this idea to inputs of a neural network layer. Although attention-like mechanisms have been used within neural network computations for decades, the wide adoption of attention in deep learning arose with neural machine translation in the last decade. There, attention mechanisms were used in combination with recurrent encoder-decoder neural network architectures to translate between languages [28][29]. Later, the Transformer neural network architecture, which has no recurrent components but instead relies on only a specific type of attention mechanism for dealing with sequential data, was proposed [30]. Transformer-style models utilizing the attention mechanisms have since become an industry standard achieving state of the art results on many sequential learning tasks, especially natural language processing, and being heralded as the most promising route to achieving AGI [3], [4].

3.1.1 General Attention Mechanism

The general attention mechanism can be informally defined as a computational mechanism which flexibly relates positions of two sequences together by calculating weights for

different positions in one sequence that describe their importance for a given position in the second sequence. Attention mechanisms allow for modeling of long-term dependencies between and within sequences which were difficult to model using other architectures such as recurrent neural networks [30].

Many different types of attention mechanisms for neural networks have been proposed but, for the most part, they can be generalized to a computation on a query vector $q \in \mathbb{R}^{d_q}$ and two sets of vectors called keys $k_i \in \mathbb{R}^k$ and values $v_i \in \mathbb{R}^v$. The key and value vectors are extracted from the input features of the attention mechanism. The query vector encodes the attention context and essentially tells which feature vectors the attention mechanism should pay attention to. The query vector can be designed by the experimenter or learned from data, depending highly on the context of the attention computation and the learning problem it is applied to.

Algorithm 1 General Attention Mechanism

Input:

```

 $q$ 
 $k_1, k_2, \dots, k_t$ 
 $v_1, v_2, \dots, v_t$ 
for  $i$  in  $\{1, \dots, t\}$  do
   $s_i = \text{score}(q, k_i)$ 
 $s_1, \dots, s_t = \text{align}(s_1, \dots, s_t)$ 
 $c = \text{sum}(s_i \cdot v_i)$  for  $i \in \{1, \dots, t\}$ 
return  $c$ 

```

The goal of the attention mechanism is to calculate a context vector as a weighted sum of the value vectors where the weights correspond to the importance of the information in the key vector with respect to the query vector being processed. The attention weights are calculated by scoring the query against the set of keys using a scoring function and normalizing the results to the interval $[0, 1]$ using an alignment model. The names for the vectors query and key are borrowed from information retrieval where the task is to return the keys which are the most similar to a given query [31]. The general attention mechanism is described in algorithm 1. A special case of the general attention mechanism is self-attention, where the attention mechanism is applied to two copies of the same sequence.

3.1.2 Attention in Neural Networks

As mentioned, the usage of attention mechanisms in neural networks originates to recurrent neural networks in language modeling applications. In [28], an attention mechanism was used to enrich the calculations of a bidirectional encoder-decoder recurrent neural network in the decoding phase. The attention mechanism allows for the decoder to pay attention to every position of the input sequence when decoding the encoded input. First,

a bi-directional encoder RNN is used to transform an input sequence consisting of word embedding vectors $(x_1, x_2, \dots, x_{T_x})$ into so-called annotations $(a_1, a_2, \dots, a_{T_x})$ by concatenating the hidden-states of both of the bi-directional forward passes. As a result, the annotations encode information about the preceding and following words in the input sequence in addition to the word itself. An attention mechanism was then applied when decoding position t of the output sequence by using the annotations as both the key and value vectors and the previous decoder hidden state s_{t-1} as the query. The context vector thus becomes a combination of all the relevant information in the input sequence regarding the decoding of position t of the output sequence. The attention model applied uses a trainable feed forward neural network as the scoring function, taking as input the annotation a_j and the query s_{t-1} to calculate the attention weight α_j . The attention weights are aligned using the softmax function in Equation 2.2 and the resulting context vector $c = \sum_{i=1}^{t-1} \alpha_i a_i$ is then utilized in decoding the output. This type of encoder-decoder attention has later been the standard way of utilizing attention mechanisms in sequential learning tasks. The terms additive attention or Bahdanau attention, after the author, were later coined for the attention mechanism that utilizes a simple feed forward neural network as the scoring function and alignment model.

Different variations and extensions of additive attention have been proposed for encoder-decoder networks, mostly differing in the alignment model used. Multiplicative attention replaces the feed forward network alignment model with a simple matrix multiplication qWk_i that is normalized with the softmax function. The matrix W consists of trainable parameters. Multiplicative attention has been shown to be of similar theoretical complexity as additive attention but more efficient to implement [30]. Multiplicative attention has further been simplified to dot product attention where W is assumed to be an identity matrix and thus the computation is the dot product of the inputs [32]. Dot product attention can be defined as

$$A(q, k_1, \dots, k_t, v_1, \dots, v_t) = \sum_{i=1}^t \frac{e^{q \cdot k_i}}{\sum_{j=1}^t e^{q \cdot k_j}} v_i, \quad (3.1)$$

The same formulation can be extended for multiple queries at a time by using a matrix version of the above equation. A scaled version of the dot-product attention where the scaling is done by dividing the dimensionality of the keys has become the most used attention mechanism as of late with its adoption in Transformer neural networks. The purpose of the scaling is to make the gradients more stable for the optimization algorithm used for training the Transformers neural network [31][32].

3.2 Transformer Architecture

The Transformer is an attention-based neural network architecture for processing sequential data. The generic form of a Transformer model is an encoder-decoder network with l layers in both the encoder and the decoder. The encoder and decoder layers consist of a self-attention mechanism as well as a fully connected subnetwork. The learning task of an encoder-decoder Transformer is to generate the output sequence given the input sequence and it has been used in applications such as natural language translation.

The computational flow in a Transformer neural network can be described as a process consisting of multiple steps computed position-wise for the sequence. First, given a sequence of values $S = (s_1, s_2, \dots, s_n)$, the value in each position i of the input sequence is transformed into a length d_{model} embedding vector $e \in \mathbb{R}^{d_{model}}$ using a trainable embedding layer. For a natural language processing scenario with a dictionary of size $|\mathbb{D}|$, the input sequence consists of binary dictionary index vectors and the embedding layer consists of a single parameter matrix of size $|\mathbb{D}| \times d_{model}$. A fixed positional encoding p of the same dimensionality d_{model} is then summed to the input embedding. The role of the positional encoding is to feed information about the relative positions of the input embeddings to the model. Without the positional encoding, the transformer would be agnostic to sequence order as it contains no recurrent or convolutional components. A typical positional encoding consists of summed sine and cosine functions of different frequencies. Some implementations of Transformers incorporate a trainable position embedding layer instead of a fixed encoding [30].

The summed input embedding and positional encodings are fed through the encoder layers. The encoder layers encode the inputs position-wise into a latent representation space later utilized by the decoder. The output of an encoder layer is of the same form as its input, a sequence of vectors of length d_{model} . So-called residual connections are also included such that input of each self-attention mechanism and fully connected network is summed to its output before feeding it to the following layer. The outputs of each layer are normalized using layer normalization described in [33]. As a result of the attention mechanism, the vector representation of a given position in the encoded inputs contains information from all relevant positions of the input sequence [27].

The decoder consists of similar components as the encoder but additionally applies an encoder-decoder attention mechanism to the outputs of the encoder in a way similar to the usage of Bahdanau attention in the RNN decoder in [28] described earlier in this chapter. This allows for the Transformer to flexibly focus on the most important information in the input sequence when decoding the output. The self-attention mechanism in the decoder is calculated only with respect to the output embeddings for positions $1, 2, \dots, t-1$ when decoding position t of the output sequence in order to prevent future information of the output sequence from affecting the calculations. The outputs of the last encoder

layer are then transformed back into a sequence by applying a position-wise softmax function to them. For a natural language processing application the process is equivalent to performing a multi-class classification with $|\mathbb{D}|$ classes for each position separately.

3.2.1 Attention in Transformers

The Transformer neural network uses scaled dot product attention described in Equation 3.1.2 in three different stages. The encoder and decoder layers utilize self-attention and additionally the decoder layer performs encoder-decoder attention. The encoder and decoder layers performs a self-attention mechanism to their inputs in an equivalent way. For an input sequence of s input vectors of length d_{model} , each of the s input vectors are multiplied by three trainable weight matrices W_q , W_k and W_v of dimensionality $d_{model} \times d_q$, $d_{model} \times d_k$ and $d_{model} \times d_v$, respectively. The input vectors are either the summed embedding and positional encoding vectors or the outputs of a previous layer. The resulting vectors $q_i \in \mathbb{R}^{d_q}$, $k_i \in \mathbb{R}^{d_k}$, $v_i \in \mathbb{R}^{d_v}$ for $i \in 1, \dots, s$ are then used as input for the scaled version of the dot product attention function in equation (3.1.2) which is applied separately for each position. Specifically, the output of the attention mechanism for position i is $A(q_i, k_1, \dots, k_s, v_1, \dots, v_s) \in \mathbb{R}^{d_v}$. In essence, the attention mechanism scores the query of position i against the keys of each position, in order to find which positions of the sequence the model should pay most attention to [30].

The transformer model uses h different sets of trainable matrices to calculate h attention outputs in parallel. The outputs are then concatenated to yield an output vector of length $d_v h$ which is then further projected to a vector of length d_{model} using a trainable projection matrix W_{pool} of size $d_v h \times d_{model}$. This methodology is known as multi-head attention and the individual projection-attention combination as an attention head. The same weight matrices are used for each position in a single attention head, but the different encoder and decoder layers use different weight matrices in their attention heads. The d_{model} sized vectors are then summed to the original inputs from the residual connection and fed to a position-wise feed forward network with one hidden layer and ReLU activation function. The feed forward network consists of the weight matrices W_{FF1} of dimension $d_{ff} \times d_{model}$ and W_{FF2} of dimension $d_{model} \times d_{ff}$. Each encoder and decoder layer has its own feed forward network and no parameters are shared. All described weight matrices W_q , W_k , W_v , W_{pool} , W_{FF1} and W_{FF2} have corresponding bias vectors which are added to the transformed inputs. Figure 3.1 depicts an illustration of a transformer layer and the corresponding parameters.

As stated before, the self-attention mechanism in the decoder is masked in a way that the decoder can only pay attention to previous positions. In practice, the attention function is then calculated as $A(q_i, k_1, \dots, k_{t-1}, v_1, \dots, v_{t-1})$ for position $i < t$ when decoding position t . The encoder-decoder attention in the decoder layers functions otherwise similarly,

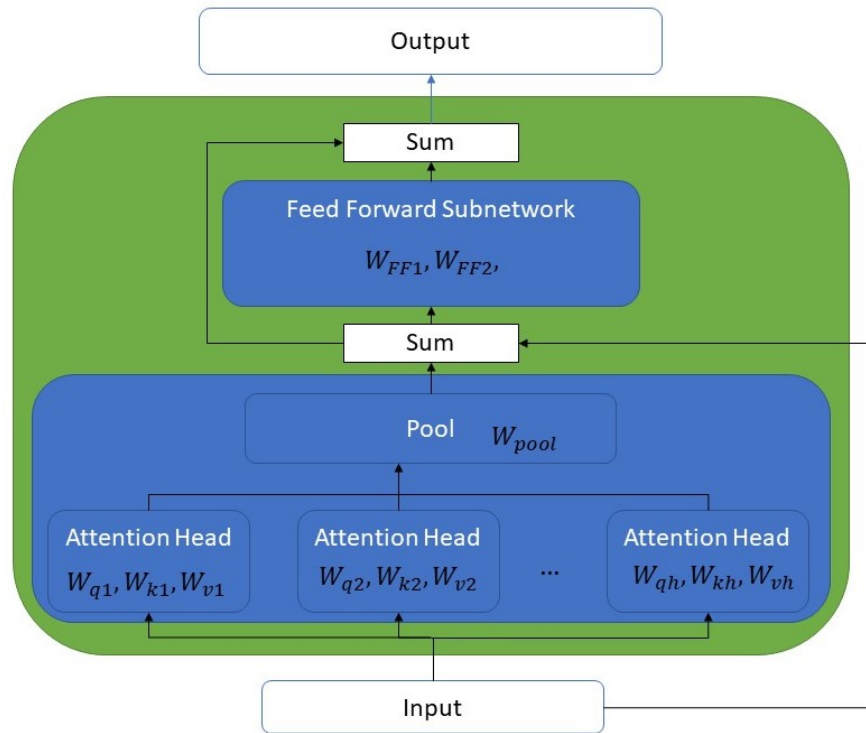


Figure 3.1. An illustration of an encoder layer of a Transformer.

but the keys and values are transformed from the outputs of the last encoder layer. The query vectors are transformed from output vectors of the self-attention mechanism of the decoder layer in question.

Transformer based models can be trained for supervised learning tasks in the same way as other neural networks, using gradient-based optimization techniques. For minibatch gradient descent, the sequence lengths of the input sequences in a single batch must be equal or padded to the size of the longest sequence. Special padding-tokens are often used for this by concatenating them to the either end of the sequences. The maximum sequence length is often globally restricted when training transformer based models, as the computational complexity of the attention mechanism grows quadratically with respect to the input length [34]. One benefit of training transformer-based models is that multi-head attention is highly parallelizable. The forward and backward passes of the different attention heads can be calculated independently of each other, which allows for simple utilization of multiple independent computational units when training or in inference. This has allowed scaling Transformer models to hundreds of billions of parameters [3].

3.3 Bidirectional Encoder Representations from Transformers

Bidirectional Encoder Representations from Transformers (BERT) is a bidirectional language model based on the Transformer encoder [27]. BERT utilizes pre-training, a type of

transfer learning where the model is first optimized to a self-supervised learning task on a large set of unlabeled data and then finetuned on a task specific labelled dataset. The representations learned by the Transformer encoder constitute an useful basis for natural language understanding, which is made evident by BERT and its various extensions achieving state-of-the art results on multiple tasks such as text classification and question answering.

BERT uses the bidirectional context of words when making inferences which allows it to gain better performance in tasks such as token classification when compared to unidirectional Transformer models of the same size, such as the Generative Pretrained Transformer (GPT). The basic implementation of BERT is pretrained using two learning tasks: Masked Language Modeling (MLM) and Next Sentence Prediction (NSP). The pretraining tasks encode general aspects of language understanding into the parameters, which can then be finetuned on any task with natural language input [27].

3.3.1 Architecture

The architecture of BERT consists of the Transformer encoder and a task specific subnetwork. Different implementations of BERT have different hyperparameters such as the number of layers l , number of attention heads per layer h , size of the hidden layer of the feed forward subnetworks d_{ff} and the size of the embedding vectors d_{model} . BERT itself does not have a decoder, as it is designed to only extract text representations which can then be utilized for any task. BERT replaces the ReLU activation function in the feed forward network with a GELU for better computational stability.

In typical applications of BERT, the outputs of the transformer encoder are fed into a task specific subnetwork which performs the final computations and the transformation required by the output datatype of the task in question. For a text classification task, the subnetwork is a simple feed forward network with a softmax activation function.

The standard version of BERT is the **BERT_{BASE}** model, which is trained using hyperparameters $l = 12, h = 12, d_{ff} = 3072, d_{model} = 768$. The total parameter count of **BERT_{BASE}** is around 110 million, with about 85 million parameters in the Transformer encoder layers, 24 million in the embedding layers and the rest in the output transformation layer connecting the encoder and the task specific subnetwork.

BERT uses word piece embeddings, meaning that a piece of input text is split into tokens of a few characters, which are then transformed to embedding vectors of dimensionality 768 using a trainable embedding layer. The vocabulary consists of 30 000 unique word piece tokens. The most common BERT implementations also employ a trainable position embedding instead of the traditional position encoding used in Transformers [27].

BERT is able to represent both individual texts and pairs of texts, which is required

for tasks such as Next Sentence Prediction (NSP). In the case of a pair of sentences a special <SEP> token is inserted between sentences to form the input text and learned segment embeddings are also summed into the WordPiece and position embeddings. Additionally, a trainable <CLS> token embedding is also included in the beginning of each sequence. The activations of the final encoder layer for the <CLS> token can be used as a representation of the entire sequence in classification tasks.

3.3.2 Pretraining and Finetuning

As stated, the standard version of BERT is pretrained using two self supervised tasks, Masked Language Modelling and Next Sentence Prediction. In MLM, a fraction of the input tokens in a training text are replaced by a <MASK> token and the task is to predict the original tokens that were masked using the context of the remaining text. The goal of MLM is to learn the bidirectional context and relationships between tokens, which can be beneficial for various tasks in the finetuning stage. The task specific subnetwork in MLM takes the activations of the final encoder layer corresponding to the masked inputs and transforms them into a vector of token probabilities using a trainable transformation matrix and a softmax activation function.

In next sentence prediction, the model is given two sentences separated by the <SEP> token and the task is to predict the binary condition of whether the second sentence directly follows the first sentence in the training data. A sentence in this context generally refers to any piece of text rather than a sentence in the grammatical sense. The goal of NSP is to train the model to learn longer term relationships between sentences which can be relevant for many fine tuning tasks. The task-specific subnetwork is a simple binary classifier with a sigmoid activation function. Some proposed variations of BERT including RoBERTa [35] and DistilBERT [36] do not use NSP in pretraining, which has been found to be beneficial for some finetuning tasks.

BERT can be finetuned for a variety of natural language processing tasks by replacing the task specific subnetwork. These include tasks such as text classification, token classification, question answering and summarization. The encoder is initialized with the weights from the pretrained encoder and all weights are typically trained when finetuning for the specific task. For text classification, the task specific network is typically a simple feed forward neural network which takes the activation of the <CLS> token from the final layer of the encoder as input and transforms them to class probabilities using a softmax activation function. In token classification, a similar task specific neural network can be used to classify each token in the sequence.

In [37] it was discovered that applying further self-supervised pretraining on the finetuning task dataset can improve performance for many classification tasks. It was further noted that a careful consideration of the learning rate for finetuning the BERT encoder is nec-

essary to avoid the so-called catastrophic forgetting phenomenon, where the weights of encoder are altered too much in the beginning of the finetuning stage resulting in forgetting of the information learnt from the pre-training data. This can lead to not benefiting from the pretraining at all which typically leads to a poor performance. Using training algorithms which adjust learning rates separately for each parameter can alleviate this problem when the starting learning rate is sufficiently small.

4. NEURAL NETWORK PRUNING

In this chapter, the general theory of pruning neural networks is introduced. First, the motivations behind pruning and the origins of pruning neural networks are discussed. Then, pruning is formalized into an optimization problem. Some notable methods and frameworks of neural network pruning are also presented. For a more detailed overview of pruning methods, see for example [38]. Finally, the specifics of applying pruning to Transformer neural networks is explored.

Pruning is a model compression technique, which attempts to reduce the complexity of a machine learning model without losing performance. The word pruning generally refers to removing unwanted material from plants. In computer science, the word pruning has been used for decades in the context of tree data structures where the word referred to simplifying a tree by removing redundant connections and nodes. Pruning has seen prominent use in machine learning with decision tree algorithms where the goal is to remove parts of an already optimized decision tree to improve generalization performance (see for example [39]). This approach has later been further applied to neural networks.

As discussed in Chapter 2, feed forward neural networks can be represented as a directed acyclic graph and thus the methodology of removing redundant connections and nodes, equivalent to the one used with decision trees, can be directly applied. Pruning methodologies of neural networks tend to generalize well to different types of neural network architectures and learning tasks, as they are not dependent on the specifics of an architecture. In practice, removing components has the effect of deleting individual parameters or groups of parameters from the neural network. This reduces the parameter count of the model which can reduce its size in terms of memory and computational requirements of the hardware [7]. In some cases, pruning can also increase the generalization performance of the neural network.

Pruning a neural network involves selectively removing parameters, which will inevitably alter the outputs given by the model by some margin for specific inputs. While an unaltered, fully-optimized network might exhibit peak performance on a training dataset, pruning can enhance its generalization capabilities through reducing the model complexity. The overall aim of pruning is to identify and eliminate parameters in a manner that minimizes any negative impact on model generalization performance.

As discussed in Chapter 2, most modern deep learning architectures tend to be heavily overparametrized and also suffer from high computational requirements. This phenomenon is even more prevalent when utilizing transfer learning where models pretrained on huge datasets are fine tuned on tasks with less data. Attempting to remove excess parameters can in such cases lead to a high decrease in memory requirements and computational costs without affecting model performance for the specific task in question.

4.1 Theoretical Framework of Pruning

Let $f(x|\Theta)$ be a neural network with parameters Θ that optimize cost function c over some probability distribution of data $p(x, y)$. For the sake of this definition, it can be assumed that $\Theta \in R^{n \times m}$ is a single matrix containing all the parameters of the model, although the approach generalizes easily to multiple parameter matrices or tensors. The goal of pruning is to find a sparse parameter matrix Θ' with a minimum possible value of the cost function. For this purpose, sparsity can be defined as the amount of zero-valued parameters in the parameter matrix. Pruning can then be defined as the constrained optimization problem

$$\begin{aligned} \min_{\Theta'} \quad & c(x | \Theta') - c(x | \Theta) \\ \text{s.t.} \quad & \|\Theta'\|_0 < B \\ \text{for} \quad & p(x, y), \end{aligned} \tag{4.1}$$

where the l_0 norm in the constraint places a bound for the number of nonzero-valued parameters in Θ' [40], [38]. In the usual case, there is no direct access to the full data generating distribution, and thus the optimization is done for the empirical risk function described in Equation 2.10. The sparse reparametrization Θ' can also be represented as an elementwise product of the original parameter matrix Θ and a binary mask variable of equal dimensions M .

$$\Theta' = M \odot \Theta. \tag{4.2}$$

With this formulation, finding the optimal Θ' is equivalent to finding the optimal mask variable M . In practice, a neural network consists of multiple parameter tensors $\Theta_1, \Theta_2, \dots, \Theta_p$ of varying dimensions and corresponding masks $M_1, M_2, \dots, M_p$ are jointly optimized when pruning the model.

Another way of formalizing pruning is through the multi-objective optimization framework. There the goal is to simultaneously optimize for both the performance of the model and the amount of zero-valued parameters.

$$\min_{\Theta'} \quad (c(x | \Theta') - c(x | \Theta), \|\Theta'\|_0) \tag{4.3}$$

The benefit of this framework is that it allows for jointly optimizing for both sparsity and performance without the need to predetermine the sparsity level. By utilizing linear scalarization parameters $\lambda_{\text{performance}}$ and $\lambda_{\text{sparsity}}$, this can be further denoted as

$$\min_{\theta'} \lambda_{\text{performance}} (c(x | \Theta') - c(x | \Theta)) + \lambda_{\text{sparsity}} \|\Theta'\|_0. \quad (4.4)$$

The setup can be simplified further by setting $\lambda_{\text{performance}}$ to a constant, and optimizing for a specific choice of $\lambda_{\text{sparsity}}$ without the loss of generality.

4.1.1 Computations of a Pruned Neural Network

The forward and backward pass computations of a neural network must be altered slightly when training a pruned neural network. Let θ_j be a parameter matrix of a layer f_j in a feed forward neural network and M_j be its corresponding pruning mask. The forward pass computation of the layer is then simply

$$f_j(x|\theta_j, M_j) = (\theta_j \odot M_j)x. \quad (4.5)$$

For the backward pass, the masked gradient computation becomes

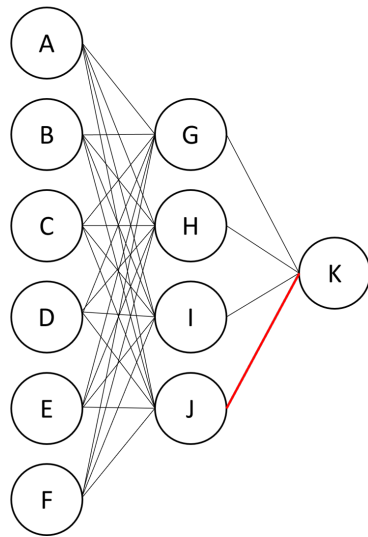
$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{\partial \mathcal{L}}{\partial \theta \odot M} \odot M. \quad (4.6)$$

This formulation can be directly extended to other types of neural network layers and articles. As an example, for a self-attention mechanism with weight matrices W_q , W_k and W_v and corresponding masks M_q , M_k and M_v , the forward pass computation of the q_i vector becomes $(W_q \odot M_q)s_i$. The process is equivalent for the key and value vectors and extends similarly to the embedding and pooling layers of Transformers.

4.2 Pruning Neural Network Structures

A single connection in a feed forward neural network represents an output of a single node A being fed as an input to the next node B with the possibly accompanying nonlinearity applied in between. Deleting one such connection would remove the need for one parameter of node B . This removing of individual parameters is known as unstructured pruning. Although unstructured pruning reduces the total amount of parameters needed and thus the memory requirements of the neural network, the effect on computational requirements is less straightforward. In practice, most modern hardware and deep learning libraries may not benefit from removing individual parameters as the implementations of the underlying tensor operations are not optimized for sparse computation [38].

Unstructured pruning of connection J-K



Structured pruning of node J

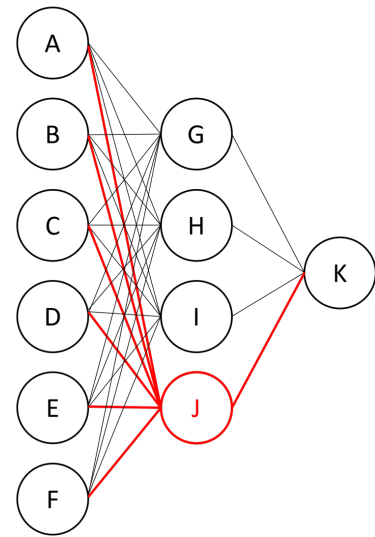


Figure 4.1. A comparison of structured pruning and unstructured pruning of a feed forward neural network

Removing entire computational units from a neural network architecture is known as structured pruning. Removing a single node from a basic feed forward neural network has the effect of removing its computation from the forward pass of a neural network and reducing the parameters needed for the computations of the next layer. If calculations of a node involve p parameters and the node is connected to n other nodes, the total reduction of parameters is $p + n$. Figure 4.1 highlights the difference of unstructured and structured pruning of a feed forward neural network. Structured pruning has the same benefits of reducing memory requirements as unstructured pruning but can also make the computations more efficient without requiring specialized hardware. When a computational unit is skipped entirely in the computation of the forward pass, the amount of tensor operations needed is directly reduced.

Structured pruning can also be thought about in terms of the parameter matrices that are used for the computations. In a fully connected scenario, pruning one node would correspond to removing a row from the parameter matrix. This type of sparsity is better handled by contemporary deep learning hardware and software when accelerating the computations. Other types of structured manipulations can also be applied, such as removing $k \times k$ blocks of parameters from a matrix. Although this does not correspond to any easily definable change to the model architecture, the benefits for accelerating the computations are similar.

4.3 Pruning Criteria

The choice of which components to prune from the neural network is the most essential part of any pruning method. This entails essentially solving the optimization problem in Equation (4.1) or Equation (4.4). Many heuristic methods have been proposed for selecting the pruned parameters. Most methods involve a scoring heuristic for individual components and a criterion which determines the pruning decision based on the scores.

The most elementary type of pruning is random pruning, where a fraction q of all of the parameters of the neural network are deleted at random. Due to its elementary nature, random pruning is often used as comparison baseline when discussing the results of different pruning methods. Random pruning can be applied globally to the entire network or layer-wise, where a constant fraction is deleted from each layer. It is also straightforward to apply a random pruning criterion for structured pruning by sampling random components instead of parameters.

In a parameter norm criterion, the parameters that are smallest in magnitude are deleted from the network. The assumption is that parameters that end up with low absolute values after training do not contribute a lot to the computation of the forward pass and can thus be deleted without a large effect on performance. A parameter norm criterion can also be applied to structured pruning by ranking the components by the norm of their parameters and removing the components that have the lowest norm, using a definition of norm suitable to the component in question. Both L1 and L2 norms have been successfully used in structured pruning. Parameter norm pruning is also referred to as zero-order pruning or magnitude pruning referring to the fact that it does not utilize gradient information.

Another established criteria for pruning neural networks is gradient magnitude pruning, also referred to as first-order pruning. Gradient magnitude pruning utilizes first order derivative information. The sizes of gradients of the parameters are stored during back-propagation and the gradient information is utilized when deciding which parameters to prune. A simple way to make the decision is to prune parameters which have gradients near zero. Intuitively, the parameters that are not changed by gradient descent are either unimportant for the loss function or such that the loss function has already achieved a minimum with respect to the parameter in question. If the latter is true, pruning the parameter might not be beneficial. Thus a hybrid criterion where both zero-order and first order information is utilized have often been found more useful. An example of this is movement pruning [41] where parameters moving towards zero are pruned.

Methods utilizing the second-order derivative of the parameters when choosing the pruned parameters have also been studied extensively for example in [42]. These methods require approximating the inverse of the Hessian matrix of the mask variable usually achieved using an empirical Fischer information matrix. Many state-of-the art pruning

methods are based on utilize second-order gradient information.

The obvious downside of gradient based pruning methods is that they only work when gradient information is available. Thus they often require a differentiable surrogate cost function to be optimized to. For many learning tasks, gradient information is not available and such criteria can not be utilized.

4.4 Pruning Strategies

The decisions which determine how and when pruning is applied to the neural network can be called a pruning strategy. Although multiple different pruning strategies have been presented in literature, a majority of them are variations of one default strategy which consists of three steps [38]. First, the network is trained on a training data set until convergence is achieved according to some convergence criterion. Next, the network is pruned using a chosen pruning method. Lastly, the pruned network is trained further on the same training set in order to allow the surviving parameters to be adjusted to the change in architecture. The training stage executed after pruning is referred to as finetuning, which is not to be confused with the same term used in the context of transfer learning. A smaller learning rate is typically used in the finetuning stage to prevent overfitting.

Algorithm 2 Iterative Pruning After Training

Input:
 n , the amount of main loop iterations
 (X, y) , the training data set
 θ , the initial parameters of the model
 Train θ on (X, y)
for i in $\{1, \dots, n\}$ **do**
 Apply pruning method to θ
 Finetune the pruned θ on (X, y)
return θ

The pruning and finetuning steps can be iterated until the desired amount of sparsity or another stopping criterion is achieved. This high level process first described in [43] is summarised in Algorithm 2. Variations of the basic algorithm include gradual pruning [44], where an increasing amount of parameters are deleted during training in order to reach a desired level of sparsity as well as different types of post training pruning frameworks that do not require further training on the whole dataset after pruning.

4.5 Pruning Transformers

Pruning Transformer neural networks follows the same basic principles and frameworks as pruning any neural networks. Both structured and unstructured pruning can be applied to both the attention heads and the feed forward networks within the BERT encoder and

decoder. The word embedding layers can also be pruned using unstructured pruning. Utilizing pruning may be especially useful for large pretrained Transformers which are finetuned for a specific downstream task. The reasoning behind this is that much of the information encoded into the parameters during pretraining may be irrelevant for a single given task. The task may involve only a small fraction of the available vocabulary and benefit only from a specific set of the model's general language understanding capabilities. Pruning provides a promising way of removing this excess information and thus making the model less heavy.

Transformers contain prunable parameters in the embedding layers, attention heads and the feed forward subnetworks. The feed forward subnetworks as well as the embedding layers can be pruned exactly a regular feed forward network would be. For the attention heads, the transformation matrices W_q , W_k and W_v related to the query, key and value vectors can be pruned in a structured and unstructured manner similar to pruning the parameter matrix of a single layer of a feed forward network. Further, the pooling matrix which is applied after concatenating the outputs of the multiple attention heads, and the word embedding matrix can be pruned in a similar fashion.

Structured pruning can also be applied to the layers by removing entire attention heads from the forward pass of the network. This removes the parameters in the query, key and value transformation matrices, leading to a reduction of $d_{model} \times d_{attention} \times 3$ parameters. Further, this removes the need for $d_{attention}$ parameters from the pooling matrix applied after the concatenation of the outputs of the attention head.

Pruning BERT follows the same pruning principles as Transformers in general. A pre-trained BERT encoder contains the same prunable parameters as a generic transformer and the task specific subnetwork can be pruned using pruning strategies relevant to the task in question after finetuning. The transfer learning utilized in BERT presents a new dimension to the pruning process. Pretraining and finetuning can be combined with the pruning and finetuning framework in 2 in multiple ways. Pruning can be applied iteratively during both the self-supervised pretraining and supervised finetuning phases or statically after pretraining and/or after finetuning. Any combination of these approaches is also possible. The potential sparsity gains for task specific pruning during or after finetuning are intuitively greater as the task in question might not require all the parameters that were utilized in the pretraining stage. Previous results on Transformer pruning are discussed in Chapter 7.

5. GENETIC ALGORITHMS

In this chapter, the mechanisms and background theory of genetic algorithms are discussed. First, an overview of search problems is given. Next, the various components that define a genetic algorithm are considered. Finally, different variations of genetic algorithms and their use cases are also discussed.

Genetic algorithms (GA) are a class of stochastic search and optimization algorithms that are based on natural selection, originally proposed by Holland in [45]. They provide a means of solving complex optimization problems with arbitrary cost functions and search spaces. Genetic algorithms search multiple directions of the search space simultaneously and thus belong to the category of population-based search methods. They provide a balanced approach with regards to exploration of the search space and exploitation of previously found promising solutions, which can be tuned by various hyperparameters.

5.1 Search Problems

Search problems involve finding an optimal solution from a space of possible solutions. A search problem is defined by a search space S containing the possible solutions and one or more goal states. A goal state can also be defined through a goal test function $G : S \rightarrow \{0, 1\}$ which determines whether a solution is a goal state. Often, the goal test is determined by a real-valued cost-function, which measures the quality of found solutions.

Search problems can be categorized into two classes: deterministic and stochastic search problems. In deterministic search problems, the outcome of the search is completely determined by the initial conditions of the system. In stochastic search problems, the outcomes are influenced by some inherent randomness or unpredictability. Factors such as the complexity of the search space, the lack of gradient or structural information, and the possible presence of multiple local optima can make search problems challenging. For these reasons, traditional gradient-based optimization methods often prove ineffective, and stochastic methods like genetic algorithms have been developed to address these complexities.

In optimization and search algorithms, there is an inherent trade-off between sufficient

exploration of the search space and exploitation of information gained from previously found solutions. Exploration refers to searching novel, untested directions in the search space. This is a vital component of any search algorithm especially in the early stages of an optimization process or when the current solutions are not satisfactory. Sufficient exploration ensures that the algorithm does not get stuck in local optima by diversifying its search, especially in large and complex search spaces.

Exploitation refers to utilizing previously found promising solutions in deciding future search directions. An algorithm with sufficient exploitation focuses on areas of the search space near the best solutions found so far. Exploitation is essential in latter stages of the search process, ensuring convergence towards a maximally optimal solution.

The trade-off between exploration and exploitation is important for the success of any search algorithm. An algorithm that explores too much without sufficient exploitation might become inefficient, wasting computational resources on less promising areas of the search space. Conversely, an algorithm which exploits too aggressively without adequate exploration might converge to a local optimum and miss out on potentially better solutions elsewhere in the search space.

5.2 Structures of Genetic Algorithms

A typical genetic algorithm consists of a set of N random initial solutions to the search problem, a genome space G into which the solutions are encoded, a fitness function $\phi : G \rightarrow \mathbb{R}$ to measure the quality of candidate solutions and various genetic operations that can be applied to known solutions to generate new solutions. An individual solution candidate c_i is referred to as a chromosome or an individual and the set of chromosomes $\mathbb{P} = \{c_1, c_2, \dots, c_N\} \in G^N$ as a population. The population evolves over time as genetic operations $\Omega : G \rightarrow G$ are applied to selected chromosomes in iterations referred to as generations. The genetic operations are functions which determine the changes that happen in genes between generations. Typically used genetic operations are mutation and crossover, which are detailed later in this chapter. They are usually defined in a way that properties of the most fit solutions, as defined by the fitness function, are passed on onto the next generation in a process reminiscent of biological natural selection. Algorithm 3 describes the general process of a genetic algorithm.

5.2.1 Gene Encoding

Genetic algorithms require that the candidate solutions can be encoded from the space of possible solutions S into a genome space G . Ideally, the mapping from S to G should be injective. In other words, no two distinct solutions should be mapped to the same genome. Typically, a genome consists of a tuple of genes $g_i \in A$ that defines all of the

Algorithm 3 Genetic Algorithm

```

Initialize a population  $\mathbb{P}_0$  of  $N$  individuals randomly
repeat
  for  $c_i$  in  $\mathbb{P}_i$  do
    Fitness evaluation: Calculate  $\phi(c_i)$ 
  repeat
    Selection: Select fit individuals from  $\mathbb{P}_i$ 
    Genetic Operations: Apply genetic operations to chosen individuals
    Add the resulting individuals to  $\mathbb{P}_{i+1}$ .
  until  $|\mathbb{P}_{i+1}| = N$ 
until convergence
  
```

information of a solution. The set A is the space of all possible values a gene can take. The value of a specific gene is referred to as an allele. The word genotype is used to refer to the encoded genetic information of a solution, contrasted with phenotype which refers to the characteristics of the solution itself. A genome of a chromosome i is defined as

$$\mathbf{g}_i = (g_1, g_2, \dots, g_n) \in G = A^n. \quad (5.1)$$

Although the genome space G can theoretically consist of arbitrary objects, typically either binary, integer or real-valued gene encodings are used. In a binary encoding the gene space is $G = \{0, 1\}^n$. The genome of a chromosome is then a string of bits. The integer and real-valued encodings have $G = \mathbb{N}^n$ and $G = \mathbb{R}^n$, respectively.

The properties of the gene encoding used can have a large effect on the performance of the entire genetic algorithm. Several properties of a good gene encoding have been proposed in [46]. Some relevant properties include the following:

1. The memory and computational requirements of chromosomes and operations on them should not be unnecessarily high.
2. Any permutation of a chromosome should map to a solution.
3. Small changes to the genotype should yield small changes in the phenotype
4. Combining parts of genotypes should yield combinations of their phenotypes

5.2.2 Crossover

Crossover refers to genetic operations $G^m \rightarrow G^h$ that combine the genetic information of a set of parent chromosomes to form offspring, resembling the biological process of sexual reproduction. The goal of crossover operators is to produce offspring that shares the qualities of its parents to provide a new search direction into the population. The parents used for the crossover are chosen using a process known as selection.

The definition of a crossover operator is highly dependent on the type of gene encoding used. For binary encodings the most notable crossover operators that have been proposed are k-point crossover and uniform crossover. These can also be applied to many other types of encodings that can be represented as a tuple of individual genes. For real-valued encodings, many alternative crossover operations have been devised, including various arithmetic and directed crossover methods.

The k-point crossover operator is a mapping $G^2 \rightarrow G^2$ in which the input genomes are cut from k points, and genes between the cutpoints are swapped between the inputs to produce the offspring. The cutpoints can be defined deterministically or chosen randomly. The simplest case, 1-point crossover, can be defined for chromosomes $a = (a_1, a_2, \dots, a_n) \in G$ and $b = (b_1, b_2, \dots, b_n) \in G$ and cross-over point m as

$$\Omega_m(a, b) = \left((a_1, \dots, a_m, b_{m+1}, \dots, b_n), (b_1, \dots, b_m, a_{m+1}, \dots, a_n) \right). \quad (5.2)$$

Figure 5.1 presents an illustration of one-point crossover for binary chromosomes.

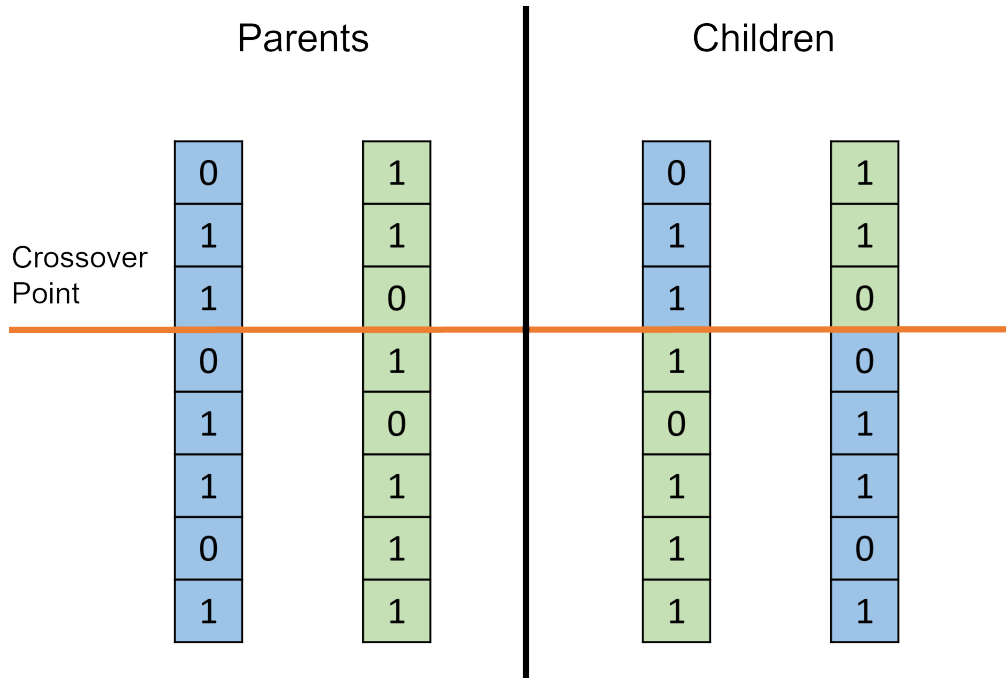


Figure 5.1. A depiction of 1-point crossover for binary chromosomes

The 2-point crossover using points m_1 and m_2 , with $m_2 > m_1$ can then be defined using function composition as

$$\Omega_{m_1, m_2}(a, b) = (\Omega_{m_2} \circ \Omega_{m_1})(a, b) = \Omega_{m_2}(\Omega_{m_1}(a, b)). \quad (5.3)$$

This can be generalized to k points $m_1, m_2, \dots, m_k$ with $i > j \implies m_i > m_j$ as

$$\Omega_{m_1, \dots, m_k}(a, b) = (\Omega_{m_k} \circ \Omega_{m_{k-1}} \circ \dots \circ \Omega_{m_1})(a, b) \quad (5.4)$$

The uniform crossover operator is a mapping $G^k \rightarrow G$ that selects genes randomly from each parent $p_1, \dots, p_k$ using mixing coefficients $\mu_1, \dots, \mu_k$. Each allele g_i of the offspring is sampled independently from the corresponding alleles of the parents using the mixing coefficients as sampling probabilities. The resulting offspring genome will then on expectation have $n\mu_i$ alleles from the parent p_i , although it is possible that the offspring is an exact clone of one of its parents.

5.2.3 Selection

Selection is the process of choosing the individuals that produce offspring for the next generation. The process can be described as a mapping $G^N \rightarrow G^k$ where k is the number of chosen individuals. The goal of selection strategies is to select individuals that will lead to offspring with higher fitness in order to focus the search on promising areas of the search space. The number of parents selected to create the next generation k is highly dependent on the crossover methodologies applied on them. In general, a low k value might lead to converging quickly to a specific area of the search space whereas a high value will maintain the diversity of solutions.

Selection methods typically include a component of stochasticity to maintain the diversity of solutions. Thus, the commonly used selection strategies map chromosomes to their probability of being included instead of mapping directly to the set of selected chromosomes. Multiple selection methods have been proposed, the most notable ones including roulette wheel selection, rank based selection and tournament selection.

In roulette wheel selection, also known as fitness proportional selection, the probability of being chosen is directly determined by the normalized fitness of the candidate solutions. For a set of chromosomes $\{c_1, c_2, \dots, c_N\}$ the probability of selecting chromosome c_i is

$$p(c_i) = \frac{\phi(c_i)}{\sum_{j=1}^N \phi(c_j)}, \quad (5.5)$$

where $\phi(c_i) > 0$ is the fitness function.

In rank based selection, the chromosomes are ranked according to their fitness and the ranks are mapped into the selection probabilities. Different types of mappings from the ranks to the probabilities can be devised. One possibility is to use exponentially decaying probabilities

$$p_r = \frac{\rho^r}{\sum_{i=1}^N \rho^i}, \quad (5.6)$$

where $\rho \in [0, 1]$ is the decay rate.

In tournament selection, a subset of chromosomes is drawn randomly from the population and a winning chromosome is selected from the subset. The winning chromosome can

be selected deterministically using the highest fitness value or stochastically as in roulette wheel selection. [47], [48] [49]

5.2.4 Fitness Evaluation

Fitness evaluation is a crucial part of the genetic algorithm. As stated before, the fitness function is a mapping $\phi : G \rightarrow \mathbb{R}$ that determines the quality of the solutions. The mapping is very general and does not need to be constrained to be a differentiable function. The usual convention is that a higher fitness corresponds to a better solution. Fitness needs to be evaluated once for each new solution created during the run of a genetic algorithm. For many applications, fitness evaluation can take the majority of the time and computational resources. Thus, the number of fitness evaluations required to reach an optimal solution is often used as a metric when discussing the quality of genetic algorithms.

When the problem being solved is an optimization problem, the objective function of the optimization can usually be directly used as a fitness function. In other cases, a careful consideration of the problem is required in order to craft a suitable fitness function that correlates well with the actual problem being solved. If the computational cost of evaluating fitness is high or the fitness function is noisy or stochastic in nature, fitness approximation can be used.

5.2.5 Mutation

Mutation refers to random changes in the genome between individuals and their offspring. They provide a way of reaching areas of the gene space that were not used in the previous generation. Mutation is essentially a mapping $G \rightarrow G$ that implements minor, usually random changes into the genome. The goal of mutation is to introduce and preserve diversity into the population of genomes. This can help the algorithm to avoid converging into local minima. A mutation operation should be designed in a way that every possible point in the gene space is accessible through mutation. Further, mutation should not introduce bias for moving towards a specific area of the gene space.

The most rudimentary mutation operation is single point mutation, where each gene has a fixed probability of being altered. For binary encoding, this is known as flip mutation, where genes are being flipped into their alternate state according to the mutation probability. One way of implementing this is to generate a value r_i from a uniformly distributed random variable $R \sim Unif(0, 1)$ for each gene $g_1, \dots, g_n$. A mutated gene then becomes $g_i^M = I(r_i < p_m) \oplus g_i$, where I is the indicator function, $\oplus$ is the exclusive or operator and p_m is the probability of mutation. Other types of mutation operations for binary gene encoding include swap mutation, which swaps the values of two genes randomly and scramble mutation, which shuffles a subset of gene values randomly.

Many other types of mutation operations have been proposed for different gene encodings. Continuous valued genes can be mutated with a single point mutation where a random value from a distribution is added to the mutated gene values. Further, permutation gene encodings and other types of restricted gene spaces have been proposed with corresponding mutation operations.

6. METHODOLOGIES

In this chapter, the genetic algorithm based pruning method for Transformers is presented. First, the theory of genetic algorithm based pruning is presented and the proposed pruning framework is described, including the gene encoding used and the fitness evaluation procedure. Next, the methodologies of the experimental case study of the thesis are examined.

6.1 Pruning with Genetic Algorithms

As discussed in chapter 4, pruning neural networks can be presented as an optimization problem where the goal is to find a sparse reparameterization for the neural network that has a minimal value of the cost function. The optimization problem is summarised in (4.4). As genetic algorithms can perform general search and optimization tasks, they can also be used for solving the optimization problem involved in neural network pruning. For pruning, the individual solutions are specific choices of the pruned parameters of the neural network in question and the fitness is measured using a combination of the sparsity of the parameters and the value of the cost function. Genetic algorithm based pruning can be seen as a type of neural architecture search done after training. There has been extensive research on using genetic and evolutionary computation methods for neural architecture search during training, such as the Neuroevolution of Augmenting Topologies (NEAT) algorithm [50]. In previous work, it has been shown neural network pruning can be solved efficiently using genetic algorithms for convolutional neural networks [51].

6.1.1 Gene Encoding for Pruning

For pruning neural networks, a standard gene encoding is a binary encoding where a single binary allele controls whether a specific component of the network gets pruned or not. This approach has been used among others in [51]. For unstructured pruning, the components are individual parameters though the same approach can be used for structured pruning as well by pruning nodes or even larger components like Transformer attention heads. For this approach, the genotype of a solution is a tuple of binary values the length of the amount of pruned components. The mask variables M controlling the pruning of specific parameter matrices are constructed out of specific genes of the solution in order

to create the final pruned network. The mask building process is described by a mapping

$$\mathcal{M} : \{0, 1\}^{|\Theta|} \rightarrow (M_0, M_1, \dots, M_m), \quad (6.1)$$

where $|\Theta|$ represents the total number of prunable components involved in the pruning process and $M_i \in \mathbb{R}^{k_i \times l_i}$ for $i \in 1, 2, \dots, m$ are the resulting m mask matrices. The parameters θ of a model pruned with an individual with the chromosome c are denoted as $\theta \mid \mathcal{M}(c)$. The mask building process of a Transformer encoder layer is depicted in Figure 6.1.

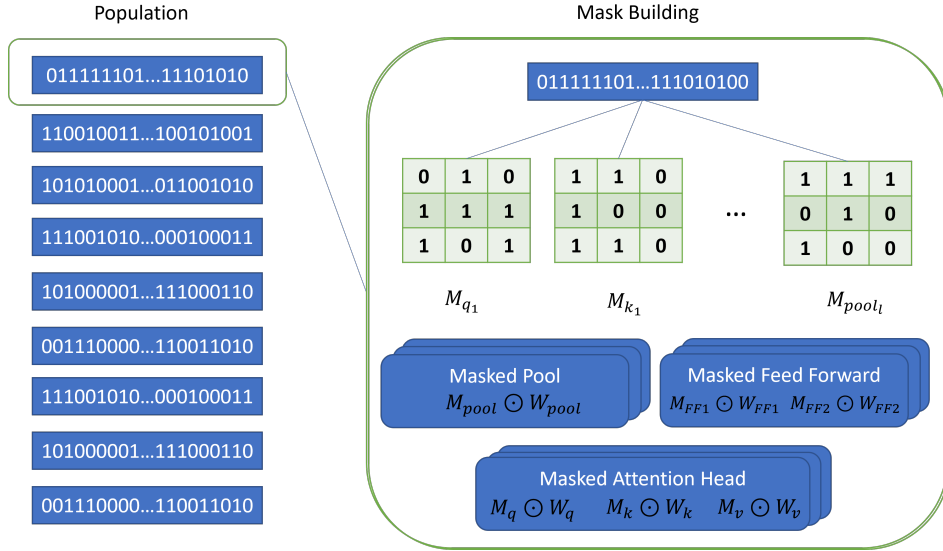


Figure 6.1. A diagram depicting the mask building process for a Transformer encoder layer

The binary gene encoding described above fits well in to the properties of a good gene encoding described in [46] and section 5.2.1. When considering the first property, the binary encoding is the simplest possible gene encoding in terms of computational complexity. The binary encoding allows for simple and efficient implementations of crossover and mutation, although the search spaces can become large for models with many parameters. With regards to the second property, permutations of solutions will always lead to some combination of pruned parameters. Most permutations lead to a functional neural network architecture and thus a valid phenotype. Though, a permutation that leads to removing an entire layer from a neural net, making its underlying graph structure unconnected, can render it practically useless. The consideration is rather straightforward for properties 3. and 4. A small change in the genotype leads to a correspondingly small change in the choice of pruned parameters. Further, any combination of genotypes will lead to some combination of the pruning choices of the individual solutions.

6.1.2 Fitness for Pruning

The fitness function for pruning a neural network should reflect the quality of the solution, that is the amount of sparsity achieved and the change in performance of the network. The multi-objective optimization framework described in (4.1) can be directly used by estimating the cost on training samples before pruning and after pruning and treating the linear scalarization parameters $\lambda_{\text{performance}}$ and $\lambda_{\text{sparsity}}$ as hyperparameters of the genetic algorithm.

The fitness function of a genetic algorithm is not required to be differentiable. Thus, if the cost function used during training of the neural network is a surrogate function for some non-differentiable primary cost function, the primary cost can be used instead in fitness evaluation directly. One example of this is using classification accuracy or the F1-score in the fitness evaluation instead of the differentiable loss function used in training of a classification model.

The fitness of the pruning solutions needs to be measured once for each new solution created. This means that in practice, fitness evaluation is the most computationally expensive component of a genetic algorithm. With this in mind, running the validation on the entire dataset may become quite computationally cumbersome for a large training dataset and population of individuals. A way to alleviate this issue is to use a smaller random sample of training examples for the validation. A drawback of this approach is the possibility of overfitting the selection of the mask variables to the specific choice of training samples used in the validation. This problem can be somewhat alleviated by using a different random sample for the validation of each generation. The fitness evaluations of the individuals of a generation can be computed in parallel, which could be an alternative solution, given suitable hardware.

6.2 Proposed Framework

The proposed genetic algorithm based pruning approach for Transformers follows the high level strategy of iterative pruning after training described in Algorithm 2, where pruning and finetuning stages are iterated until a desired sparsity level is reached. From this point forward, we will refer to this method as genetic pruning. The pruning stage consists building an initial population of binary encoded pruning solutions and evolving them through a predetermined number of generations N_G . Between the generations, standard mutation, crossover and selection operators are executed. The best performing individual is then chosen from the final generation and the pruning is applied to the Transformer model. The remaining parameters of the resulting model are then finetuned on the original training dataset for a predetermined number of epochs N_E . A completion of the entire genetic algorithm and the subsequent finetuning step completes a single iteration of the main

loop of Algorithm 2. An early stopping heuristic is applied to both the genetic algorithm and the finetuning. The entire algorithm is described in Algorithm 4.

Algorithm 4 Genetic Pruning

Input:

n , the amount of main loop iterations
 N_G , the number of genetic algorithm generations
 N_E , the number of finetuning epochs
 $\mathcal{T}_P$ the early stopping criterion for the genetic algorithm
 $\mathcal{T}_F$ the early stopping criterion for finetuning
 (X, y) , the training data set
 $\mathcal{M}$, the mask building function
 θ , the initial parameters of the model
 Train θ on (X, y) until convergence
for n iterations or until desired sparsity is reached **do**
 Calculate $|\theta|$, the amount of components not pruned in previous iterations
 Create a population P of binary chromosomes of length $|\theta|$
 for N_G generations or until $\mathcal{T}_P$ **do**
 for Each chromosome $c \in P$ **do**
 Evaluate $\theta \mid \mathcal{M}(c)$ on a random sample of (X, y)
 Apply genetic operations to create the P of the next generation
 Select the most fit individual c_{best} from the final P
 Finetune $\theta \mid \mathcal{M}(c_{best})$ on (X, y) for N_E epochs or until $\mathcal{T}_F$
return θ

For any given iteration of the main loop, the pruning process is applied to only the parameters which were not pruned in the previous iterations. As a result, the dimensionality of the chromosomes of the genetic algorithm and thus the search space are constantly decreasing, leading to an acceleration of the iterations.

Genetic pruning has some benefits when compared to traditional methods such as gradient based pruning. The genetic pruning method does not utilize gradient information and the fitness function of the genetic algorithm does not need to be differentiable. The method can thus be utilized for tasks that do not have differentiable cost functions, such as those often found in reinforcement learning. The fitness function can be any type of a real-valued function and can thus be designed for custom use cases. One example of this would be to directly optimize the amount of floating point operations used by the model instead of the number of parameters.

Genetic pruning can be directly combined with many other pruning methods by altering the initialization of the population. Typically, the chromosomes of the initial population are initialized randomly, but by initializing the population with the pruning solutions provided by the other method, the genetic algorithm can essentially perform a local search around the provided solutions. In fact, even with random initialization, the genetic pruning method is doing a local search over the solutions provided by the random pruning method.

To combine genetic pruning with magnitude pruning, described in Chapter 4, the population is initialized by a combination of random pruning solutions and magnitude pruning solutions at different sparsity levels. For predetermined levels of sparsity $s_i, i = 1, 2, \dots, r$ a randomly initialized solution with sparsity s_i is added to the population. This is achieved by sampling each gene independently from a $Bernoulli(s_i)$ distribution. Next, a solution achieved by magnitude pruning at the same sparsity level is added to the population. This is repeated until for each s_i , resulting in a total of $2i$ initial solutions of which i are random and i are magnitude pruning solutions of different sparsity. Including the magnitude pruning solutions into the initial population allows the genetic algorithm to directly search promising areas of the search space around the known solutions. The randomly initialized solutions ensure that the algorithm also attempts to find solutions from other parts of the search space, maintaining a balance of exploration and exploitation.

The relatively high computation time and memory requirements of the fitness evaluation and genetic operations can be seen as a significant drawback of the method when compared to some gradient based pruning methods. Another drawback is that there is no straightforward way of predetermining the hyperparameters of genetic pruning that lead to a specific amount of sparsity, nor is it possible to determine in advance how many iterations of the method are needed. Further, the choice of hyperparameters of the genetic algorithm adds another layer of complexity to the neural network design process for the experimenter, which can be seen as a drawback.

6.3 Experiment Details

The case study experiment consists of pruning a BERT model trained on text classification tasks using the genetic pruning method. First, a pretrained $BERT_{BASE}$ [27] model from the HuggingFace Transformers model repository [52] was finetuned on the two separate classification tasks. Then, the genetic pruning method and a baseline pruning method were applied separately to the finetuned models. The resulting loss of accuracy and level of sparsity reached were compared. Both structured and unstructured pruning versions of the method were applied on the model. The pruning methods were implemented by extending the native pruning capabilities of the PyTorch deep learning framework [53] with a custom module for genetic pruning.

In unstructured pruning, the genetic pruning method was applied to the embedding layers, attention heads and feed forward subnetworks of the BERT model. This encompasses most of the trainable parameters of the BERT-model, excluding layer normalization weights and the task-specific classification subnetwork which could lead to instabilities in the computations of the model when pruned. An individual chromosome for unstructured pruning becomes a binary tuple with length equalling the number of parameters in the specified modules. For $BERT_{BASE}$, the length is over 108 million bits and

thus the search space of the genetic algorithm consists of about $2^{108,000,000}$ parameter pruning choices.

For structured pruning, each gene was chosen to represent a single attention head within an encoder layer of the BERT architecture. As $BERT_{BASE}$ contains 12 layers with 12 attention heads each, the resulting chromosome is of length $12 \times 12 = 144$ and the search space is thus of size 2^{144} . In structured pruning, the pruning method is applied on only the attention heads of the model, limiting the maximum amount of total sparsity gained. The achieved sparsity-% is reported for both the targeted attention heads, referred to as attention sparsity, and for the entire model.

The baseline method chosen to compare the genetic algorithm based pruning with was L1-norm weight magnitude pruning. This was implemented by specifying the desired level of sparsity-% and removing the corresponding amount of parameters with the lowest magnitude from the model. The resulting accuracy was reported for multiple sparsity-% values in order to compare them to the results obtained using the proposed framework. The baseline pruning method was applied to all of the parameters when comparing to the unstructured pruning approach and only to the attention heads when comparing with the structured pruning results.

6.3.1 IMDB and AG News Datasets

The case study is executed on two well-studied benchmark text classification datasets, IMDB Sentiment Analysis and AG News classification. Utilizing benchmark datasets allows for experimenting the proposed framework in a controlled environment and places the emphasis on the method instead of the specifics of the datasets.

The AG News Topic Classification Dataset [54] is a collection of 127 600 news articles taken from AG's corpus of news articles. The AG's corpus of news articles contains over one million news articles collected by the ComeToMyHead news search engine. The news articles consist of a title and a description, which are both in English. The articles are divided into four classes according to their topic: World, Sports, Business, Science and Technology. Each class has 30 000 training samples and 1900 samples for testing, totaling to a training dataset of 120 000 samples and a testing set of 7600 samples. The goal of the classifier is to predict the topic of a news article using its summary and description.

The IMDB Sentiment Classification dataset [55] is a dataset consisting of textual movie reviews in English and their human evaluated sentiment labels. The reviews chosen for the dataset are either positive, having a rating of ≥ 7 out of 10, or negative having a rating of ≤ 4 out of 10. The reviews are divided equally into the two classes and all together form a training dataset of 25000 reviews and a test dataset of equal size. The goal is to

classify the reviews to the correct binary sentiment class using the textual review.

6.3.2 Training and Pruning Details

The validation scheme for the experiments consists of a standard train-validate-test split. The original training dataset is split 80-20 % to form the training and validation datasets. The training dataset is used for the initial training stage and the subsequent finetuning iterations. The validation dataset is used for the early stopping criterion of the initial training phase, the genetic algorithm fitness evaluation as well as monitoring of model deterioration during the algorithm. The first training iteration was continued until no further improvements are witnessed in the validation epoch. The subsequent training iterations are then ran for a single epoch. The test set is used for reporting the final performance of the model before and after the entire pruning algorithm.

BERT was finetuned on the text classification tasks using the Adam optimizer with a starting learning rate of 1×10^{-6} . The main performance metric used in evaluating the model is the standard classification accuracy

$$\text{accuracy} = \frac{\text{correct predictions}}{\text{all predictions}}. \quad (6.2)$$

However, categorical crossentropy is used as a surrogate loss function for training. Categorical crossentropy is essentially equivalent to the negative log likelihood of a class for categorical data [56] and can be defined as

$$-\sum_{i=1}^C y_i \log(\hat{y}_i), \quad (6.3)$$

where $y_i = 1$ if i is the correct class and zero otherwise, and $\hat{y}_i$ is the softmax activation for class i .

Fitness proportional roulette wheel selection and single-point crossover were applied on each generation. The multi-objective optimization framework with linear scalarization parameters described in (4.4) is used as the fitness function, the optimized quantities being traditional classification accuracy and sparsity-% of the model weights. Results are reported for multiple choices of $\lambda_{\text{sparsity}}$ while keeping $\lambda_{\text{performance}} = 1$ and the other hyperparameters as constants. The experiments were run on a single NVIDIA Tesla V100 GPU. The hidden size and the various hyperparameters controlling the genetic algorithm were chosen heuristically and others such as batch size, population size and fitness evaluation sample size were chosen due to computational constraints.

Table 6.1 lists the values of the hyperparameters used in the pruning process. These hyperparameters were kept the same for each pruning run, while $\lambda_{\text{sparsity}}$ was altered.

Table 6.1. Common hyperparameters of the pruning runs

Hyperparameter	Value
Batch size	15
Learning rate	1×10^{-6}
Epochs per finetuning iteration	1
Genetic algorithm sample size	2000
Genetic algorithm population size	12
Generations per pruning iteration	50
Pruning patience	8
Main loop iterations	30

The batch size and learning rate were used in the training iterations of the framework, which were ran for 1 epoch. GA sample size is the number of samples used in evaluating the fitness of solutions and GA population size is the number of individuals in a population. The early stopping criterion for the genetic algorithm consists of two components. The algorithm is terminated if there has been eight generations after the last improvement in fitness, or after 50 generations has passed. The total number of main loop iterations was capped at 20 iterations to limit the computational costs of the experiments.

6.3.3 Algorithm Analysis

The convergence and performance of the genetic algorithm can be analyzed by inspecting the metrics collected during the process. The metrics collected for the genetic algorithm based pruning can be divided into three categories: iteration level metrics, training metrics and generation level metrics. The iteration level metrics are metrics collected after an entire iteration of the outer loop in Algorithm 4. They allow for monitoring of the convergence of the pruning method and overfitting of the model during the pruning process. For this experimental study, the metric collected was the total sparsity of the model after the iteration, referred to as iteration sparsity.

The training metrics are collected during each finetuning phase of the genetic pruning algorithm. They describe the performance of the model on the training and validation sets for each training iteration. For this experiment, the collected metrics were the training and validation set accuracy and mean categorical crossentropy loss for the training and validation set. If $N_E = 1$, as was the case in this experiment, the training metrics can be also be viewed as iteration level metrics, as there is only one training epoch per iteration.

The generation level metrics are metrics collected for each generation of each iteration of the genetic algorithm. For this experiment, these included the sparsity and accuracy of the fittest solution in the generation and the mean sparsity and accuracy of all solutions

in the generation. The corresponding fitness values can also be calculated from the sparsity and accuracy values. The generation level metrics can be used to analyze the convergence of the genetic algorithm during one iteration of the outer loop of the pruning algorithm as well as the total improvement of the solutions between iterations.

7. RESULTS

In this chapter, an experimental case study of utilizing genetic algorithm pruning for BERT is presented. The aim of the experiment is to investigate the viability of the proposed genetic pruning method by examining its performance on benchmark datasets and comparing to a baseline method. To achieve this, a BERT model was trained on text classification tasks using two separate benchmark datasets, AG News Topic Classification [54] and IMDb Sentiment Classification [55], and the genetic algorithm pruning framework described in chapter 6 was applied for both structured and unstructured pruning. The hold out test dataset classification accuracy of the model before and after pruning as well as the reached sparsity level were measured. The obtained results were compared with a baseline pruning method, described in the previous chapter, in order to confirm the feasibility of the approach.

First, the overall results for both datasets are presented. Then, a deeper examination of the pruning experiments for both datasets is presented. Finally, the strengths and limitations the proposed framework are discussed. Possible further research directions for genetic algorithm based model compression are highlighted.

7.1 Overall Results

The overall results of the experiments consist of the hold out test set performances of the BERT model after pruning for both datasets and both structured and unstructured pruning. The results discussed here are those reached after running the full 30 iterations of the genetic pruning method.

Table 7.1 summarizes the main metrics for the unstructured pruning approach for both datasets. The results are presented for the best pruning round out of four runs, each executed with different values of $\lambda_{\text{sparsity}}$, the weight of sparsity in the linearly scalarized fitness function. The best run was chosen according to the highest sparsity reached in the end of the run. The sparsity percentage of the best run is denoted as σ . For comparison, the hold out test dataset accuracy reached after the original finetuning before any pruning is denoted as a_{orig} and the final test set accuracy after pruning as a_{pruned} . The baseline accuracy is the accuracy of the model after applying the baseline pruning method at the sparsity-% achieved by the genetic algorithm based pruning method. This is denoted

as a_{baseline} . For both datasets, the unstructured genetic pruning method found a better solution than the baseline method, in terms of accuracy retained at the sparsity level reached by genetic pruning. The genetic pruning method was able to retain most of the accuracy, while the baseline performance reduced to essentially random guessing.

Table 7.1. Unstructured Pruning Results

Dataset	$\lambda_{\text{sparsity}}$	a_{orig}	a_{pruned}	a_{baseline}	σ
IMDB	1.3	90.6 %	83.7 %	48.2 %	78.0 %
AG News	1.0	94.3 %	91.1 %	28.6 %	71.6 %

Table 7.2 presents the primary results for structured pruning. In addition to the metrics described for unstructured pruning, the percentage of attention heads pruned is denoted as $\sigma_{\text{attention}}$. The total sparsity level reached is denoted as σ like before. As the attention head parameters make up about 20 % of the parameters of the model, that is also the maximum amount of total sparsity reachable by the structured pruning method. The structured pruning procedure was able to remove up to 135 of the 144 attention heads in the model resulting in only a slight drop of accuracy. Similarly to the the unstructured pruning results, the structured pruning method was also able to find a better pruning solution than the baseline method for both datasets.

Table 7.2. Structured Pruning Results

Dataset	$\lambda_{\text{sparsity}}$	a_{orig}	a_{pruned}	a_{baseline}	σ	$\sigma_{\text{attention}}$
IMDB	0.5	90.6 %	84.0 %	49.3	18.4 %	93.8 %
AG News	0.7	94.3%	91.6%	24.2	18.4%	93.8%

7.2 AG News

As mentioned, both the structured and unstructured genetic algorithm based pruning methods were able to find a better solution than the baseline magnitude pruning for the AG News dataset, after running the full 30 iterations. From table 7.1, we can see that the unstructured pruning approach with $\lambda_{\text{sparsity}} = 1.0$ found a solution with a sparsity of 71.6 % and an accuracy of 91.1 %, corresponding to a 3.2 percentage point decrease. The baseline method dropped down to a random guess accuracy for a similar sparsity level. The results were similar for other choices of $\lambda_{\text{sparsity}}$ with achieved sparsity levels ranging from 65.3 % to the mentioned 71.6 % and retained accuracies ranging from 91.0 % to 91.7 %. The structured pruning method also found a better solution in terms of accuracy retained and attention sparsity reached. The method was able to retain most of the accuracy at the attention sparsity of 93.8 % whereas the baseline method dropped down to random guess accuracy.

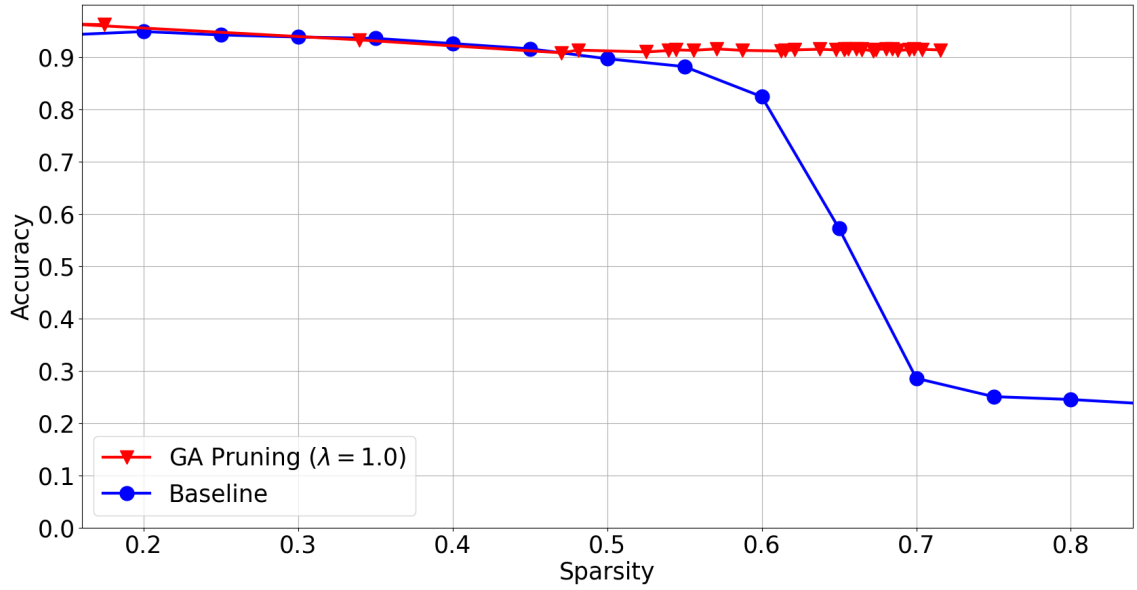


Figure 7.1. Line chart comparing the accuracy-sparsity trade-off of the baseline method to the unstructured pruning approach for AG News with $\lambda_{sparsity} = 1.0$.

Figure 7.1 presents a comparison of the accuracy-sparsity trade-off of the baseline method and the unstructured pruning approach for AG News. The figure shows test set accuracies for the baseline method measured at sparsity intervals of 0.05 and accuracies measured in the different iterations for the genetic algorithm based pruning. For this figure and all similar subsequent figures, the curve of the genetic pruning method contains values only up to the maximum sparsity levels reached by the method, in this case 71.6 %. The accuracy in the figure is specifically the validation set accuracy of the genetic algorithm based pruning method measured after the finetuning epoch of the iteration which reached the corresponding sparsity. The accuracy of the baseline method is evaluated from the entire test set for all sparsity levels.

From Figure 7.1, we can see that for lower levels of sparsity, the unstructured pruning method and the baseline method achieved similar levels of performance. The baseline method starts to show a steep drop in accuracy for sparsities higher than around 0.45. The genetic algorithm approach does not showcase a similar drop for this dataset and is able to retain most of the accuracy for the higher levels of sparsity. The points on the line representing the results of the genetic algorithm based pruning get grouped closer together for the later iterations due to the sparsity gains per iteration getting smaller.

Figure 7.2 shows a similar comparison for the structured pruning approach. In this plot, the accuracy-sparsity trade-off is presented with regards to the sparsity of only the attention heads. The accuracy of the baseline method suffers a similar steep drop in accuracy at around 80 % attention sparsity. For the genetic algorithm based pruning method, the accuracy remains stable until the final iteration and the attention sparsity reaches the mentioned level of 93.8 %.

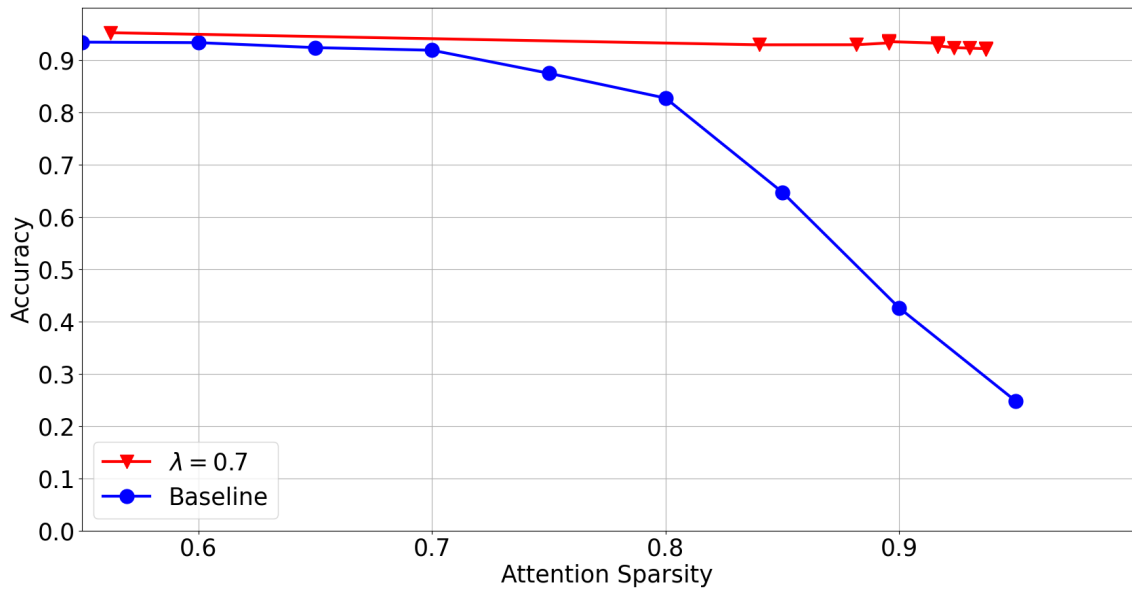


Figure 7.2. Line chart comparing the accuracy-sparsity trade-off of the baseline method to the structured pruning approach for AG News with $\lambda_{\text{sparsity}} = 0.7$.

Figure 7.3 presents the generation level metrics of fittest individual for unstructured pruning on the AG News dataset with $\lambda_{\text{sparsity}} = 1.0$. The accuracy in this plot and of the subsequent plots of generation metrics, refers to the accuracy measured for the individual in fitness evaluation, which is measured from the subset of the validation set. From the figure, it can be seen that sparsity of the fittest individual in the generation increased steadily while the accuracy was staying relatively constant. The fitness function follows a similar curve as the sparsity, since it is a linear combination of the two mentioned values. This continues to the very end of the run, which might suggest there could be room for more improvement given longer computing time and even more iterations. Though, as this analysis is based on the validation set accuracy, it does not take into consideration the effect of possible overfitting during the finetuning stages of the iterations. However, the overall results show that for these pruning runs, overfitting is not affecting performance too much as the final test accuracy is on a similar level to the final validation accuracy measured.

By analyzing the mean generation level metrics of the population for the same run in Figure 7.4, we can gain insights into the entire population across the generations. Although the boundaries between iterations are not directly visible in the figure, we can see that the iterations tend to start with a pool of sparse, low accuracy individuals due to the random initialization of the genes in the population. The search starts to then get directed towards solutions that retain the training set accuracy and improve the sparsity from the previous iteration. The solutions finally tend to converge to a high accuracy and low to moderate sparsity increase solutions. This sheds some light on the dynamics of the exploration-exploitation trade-off happening during one iteration of the search, al-

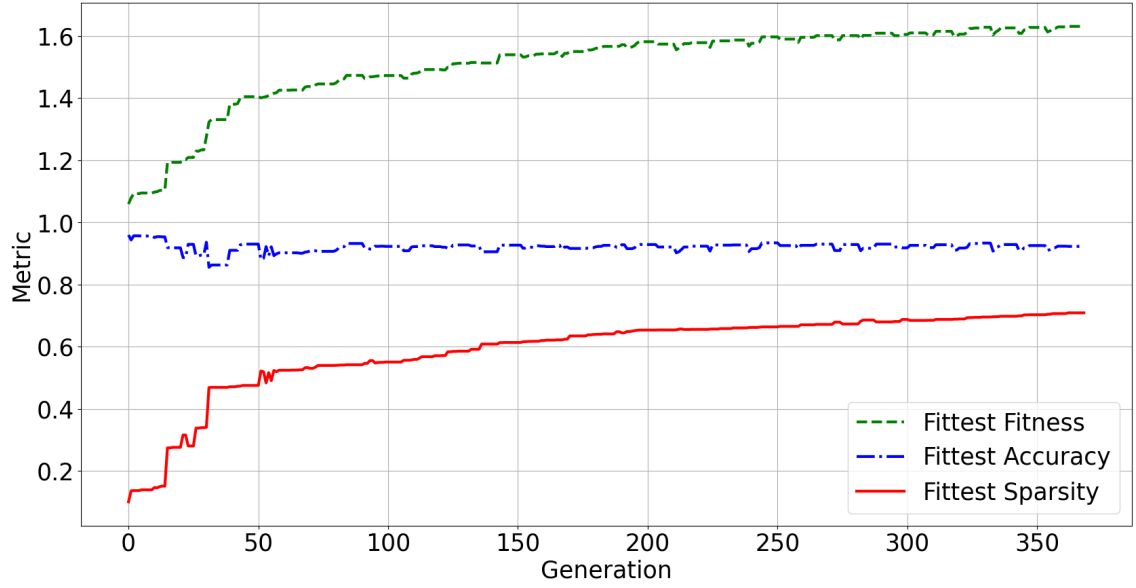


Figure 7.3. Line chart of the generation fittest metrics for unstructured pruning for the AG News dataset with $\lambda_{\text{sparsity}} = 1.0$

though more detailed distributional and similarity metrics would be required for a thorough analysis of the process.

Figure 7.4 makes apparent that the increases in sparsity per generation are getting smaller as the process goes further. This is quite intuitive as it becomes increasingly difficult to find redundant parameters when the number of parameters gets smaller. A possible way of combating this would be to make $\lambda_{\text{sparsity}}$ an increasing function of the iteration index, making the fitness function biased to more sparse solutions in the later iterations.

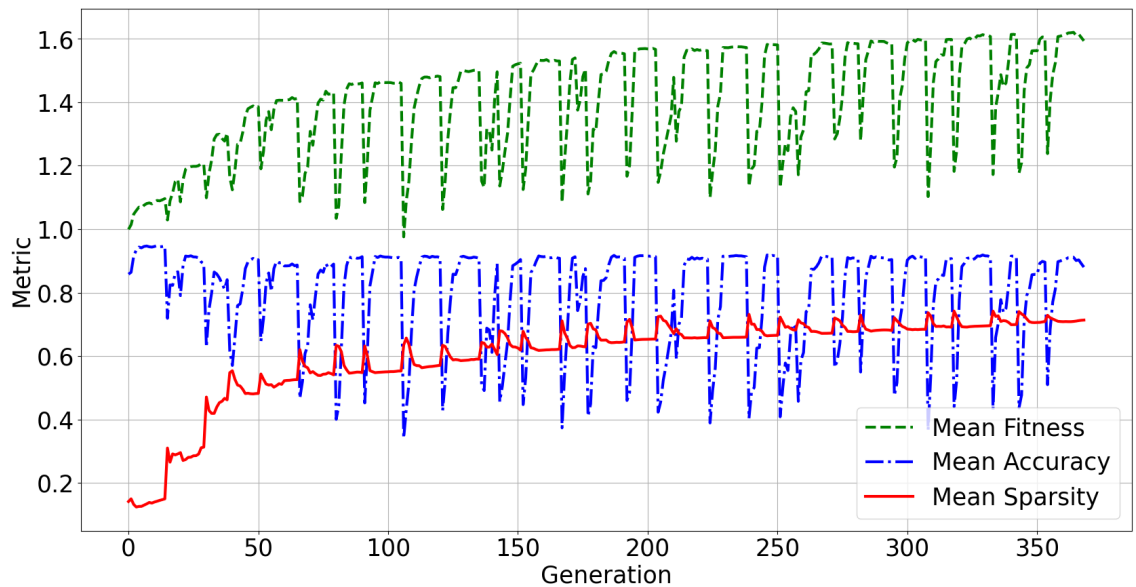


Figure 7.4. Line chart of the generation mean metrics for unstructured pruning for the AG News dataset with $\lambda_{\text{sparsity}} = 1.0$

Figure 7.5 shows the iteration level metrics for the same run. The accuracy in this plot is the validation set accuracy measured after the finetuning step of the iteration. From Figure 7.5, we can see that the validation set accuracy is staying relatively level after dropping slightly in the very first iterations. The sparsity on the other hand keeps on rising quite steadily, further implying room for more improvement. The method reaches a sparsity of nearly 50 % after a few iterations, after which the gains in sparsity become smaller.

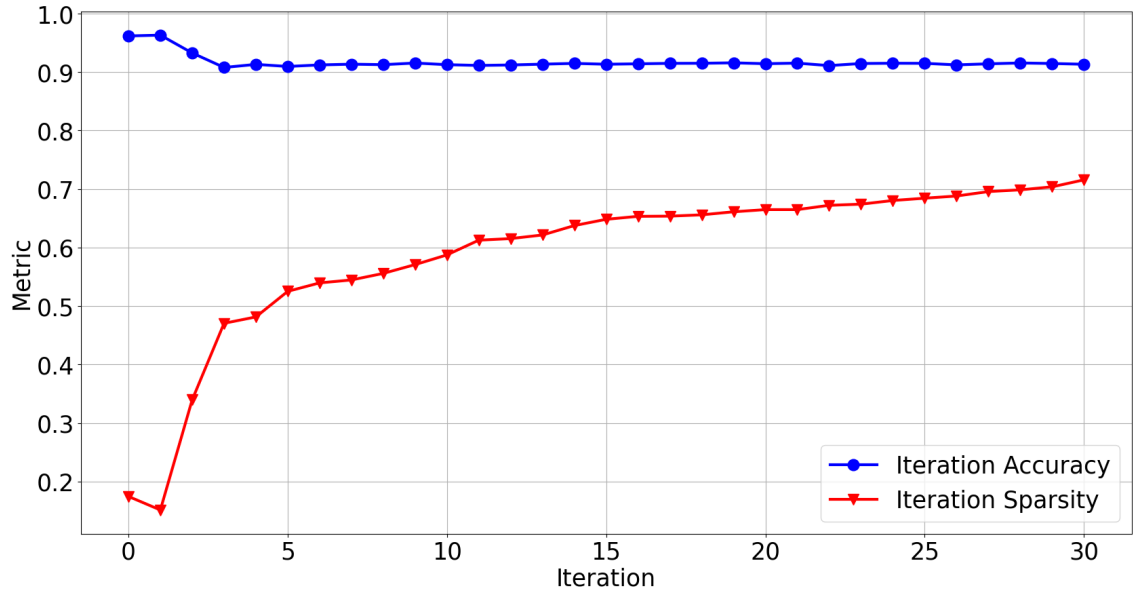


Figure 7.5. Line chart of the iteration accuracy and sparsity, measured after the finetuning step, for unstructured pruning for the AG News dataset with $\lambda_{\text{sparsity}} = 1.0$

By inspecting the training set accuracy across the iterations, which is not included in Figure 7.5, we observe that the train set accuracy increases slightly in the first iterations, and then stays relatively constant at a slightly higher level than the validation accuracy. This suggests that the decrease in accuracy during the iterations can be attributed to slight overfitting during the finetuning epochs, an issue which any pruning method which contains additional training can suffer from.

7.3 IMDB

For the IMDB Sentiment Classification dataset, the genetic algorithm based pruning method was also able to find a better solution than the baseline magnitude pruning. From table 7.1, we can see that the unstructured pruning approach with $\lambda_{\text{sparsity}} = 1.0$ found a solution with a sparsity of 78.0 % and an accuracy of 83.7 %, corresponding to a 6.9 percentage point decrease. The baseline method again dropped down to a random guess accuracy for a similar sparsity level. The results for different values of $\lambda_{\text{sparsity}}$ varied a bit more than those of the AG News runs, with achieved sparsity levels ranging from 48.2 % to the mentioned 78.0 % and retained accuracies ranging from 83.7 % to 86.0 %.

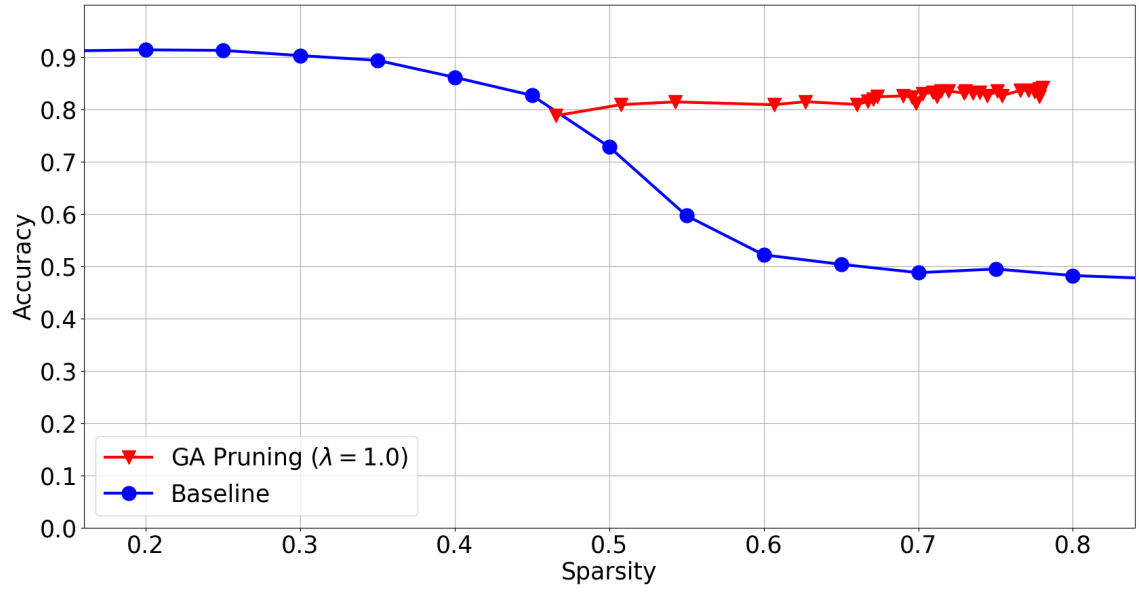


Figure 7.6. Line chart comparing the accuracy-sparsity trade-off of the baseline method to the unstructured pruning approach for IMDB Sentiment Classification with $\lambda_{\text{sparsity}} = 1.0$.

Figure 7.6 presents a line chart comparing the accuracy-sparsity trade-off of the baseline method and the unstructured pruning approach for IMDB with $\lambda_{\text{sparsity}} = 1.0$. From Figure 7.6, we can see that the pruning method starts by pruning quite aggressively, reaching nearly 50 % sparsity after the first iteration. The accuracy is then kept relatively even, while the sparsity reaches nearly 80 %. The baseline method again starts to show decline in performance at around 40 - 50 % sparsity, dropping to random guess level by around 65 % sparsity.

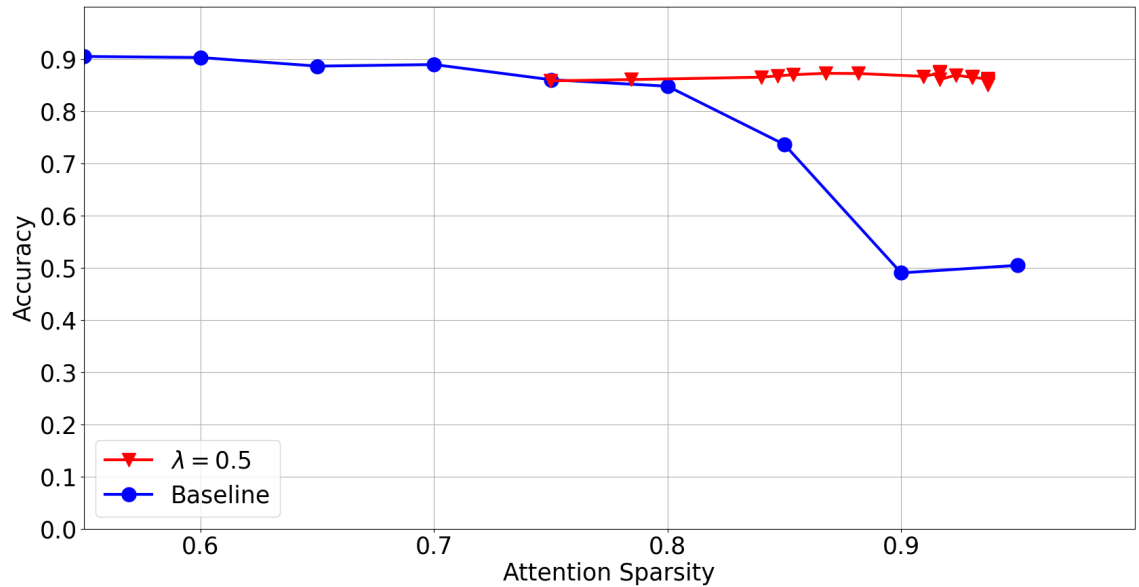


Figure 7.7. Line chart comparing the accuracy-sparsity trade-off of the baseline method to the structured pruning approach for IMDB Sentiment Classification with $\lambda_{\text{sparsity}} = 0.5$.

Figure 7.7 presents the accuracy-sparsity trade-off for the structured pruning method for IMDB with $\lambda_{\text{sparsity}} = 0.5$. The results are similar to the structured pruning results for AG News. The baseline method starts to show a steep loss in accuracy at around 80 % attention sparsity, while the genetic pruning method retains the accuracy until the final iteration.

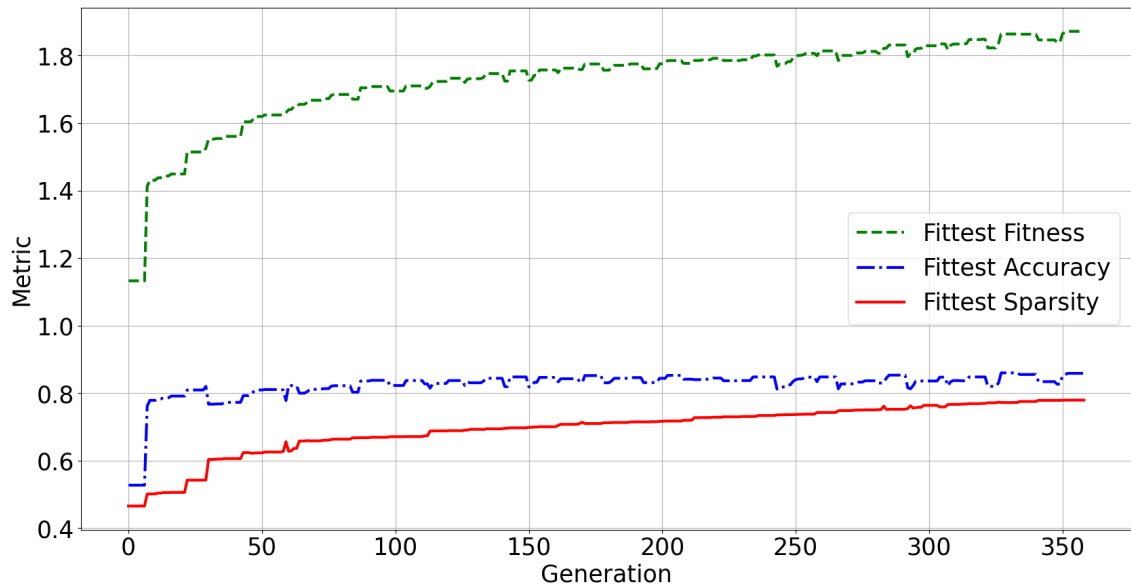


Figure 7.8. Line chart of the generation fittest metrics for unstructured pruning for the IMDB dataset with $\lambda_{\text{sparsity}} = 1.3$

Figure 7.8 presents the generation level metrics of fittest individual for unstructured pruning on the IMDB dataset with $\lambda_{\text{sparsity}} = 1.3$. A similar steady increase of the sparsity level and stability of the accuracy is also reported for IMDB. There also seems to be a continuing improvement in the later iterations, suggesting that room for improvement is also present here.

The mean metrics presented in Figure 7.9 for the same run tell a similar story of population dynamics to the AG News run. The first iteration shows different behavior though, as it first leads to a pool of highly sparse solutions with nearly random guessing accuracy, before stabilizing to a slightly more moderate level of sparsity.

Figure 7.10 shows the iteration level metrics for the discussed run. This figure also highlights the aggressive pruning happening in the first iteration as it has the lowest accuracy measured in any iteration. The accuracy increases slightly in the next 10-15 iterations, suggesting that the finetuning steps are able to make up for the accuracy lost in the first iteration. Similarly to the AG News run, the first 5 iterations showcase quick gains in sparsity which then become smaller and quite constant for the subsequent iterations.

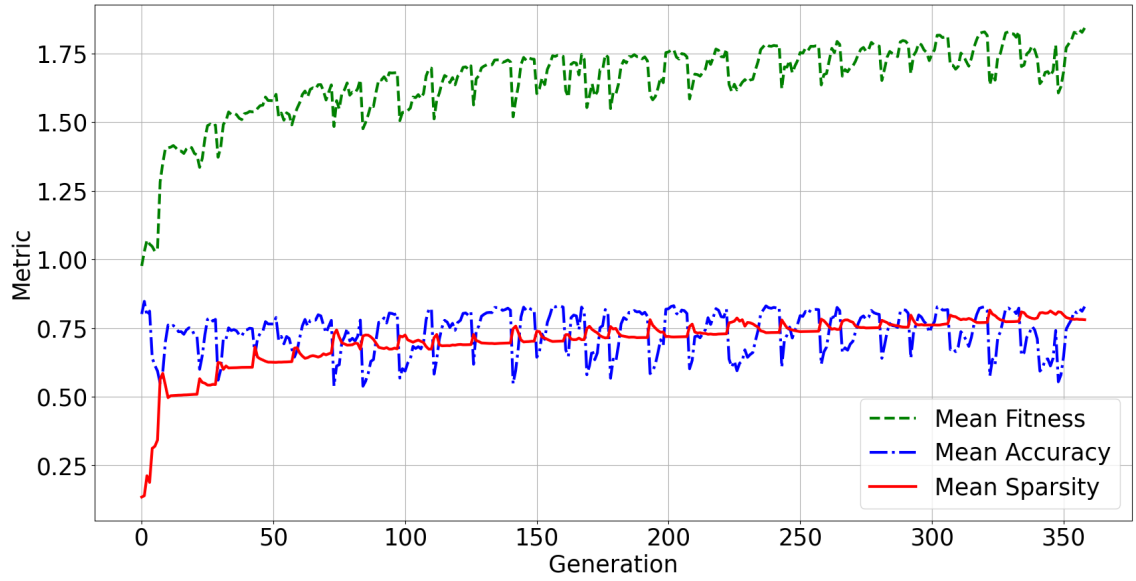


Figure 7.9. Line chart of the generation mean metrics for unstructured pruning for the IMDB dataset with $\lambda_{sparsity} = 1.3$

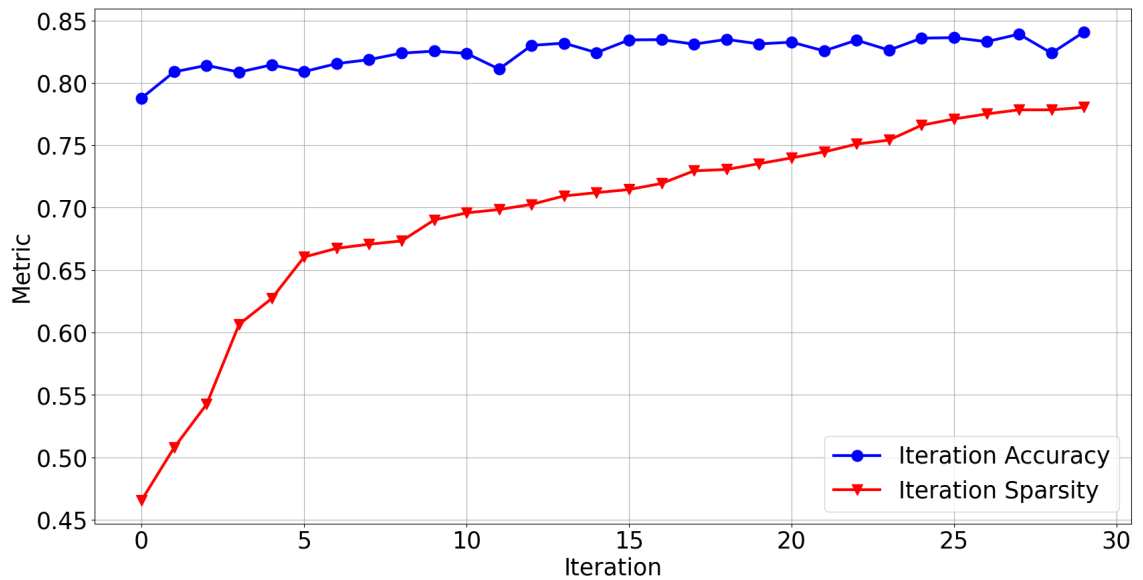


Figure 7.10. Line chart of the iteration accuracy and sparsity for unstructured pruning for the IMDB dataset with $\lambda_{sparsity} = 1.3$

7.4 Discussion

The presented results of the experimental case study provide some empirical evidence for the utility of genetic algorithm based pruning. The method achieved significantly better results than the baseline magnitude pruning for both of the datasets and with both structured and unstructured pruning. The results also seem to show room for improvement even with an identical setup and longer compute times. The genetic pruning method should also scale effortlessly to increased computational resources by increasing population size and running the algorithm for more iterations. Thus, to answer the research

question detailed in Chapter 1, it can be claimed that genetic algorithm based pruning can provide a feasible option for Transformer compression. The experimental setup provided in this thesis is relatively narrow and there are definite limitations to it, some of which are discussed later in this section. Broader and more detailed benchmarking with higher computational resources would be beneficial for more detailed analyses.

7.4.1 Comparison to Previous Results on Transformer Pruning

Many different types of unstructured and structured pruning strategies have been applied to Transformers and more specifically, language models like BERT. Most state-of-the-art Transformer pruning methods found in the literature are based on some variation of gradient based pruning. Typically, these methods are evaluated on standard language understanding benchmarks such as the Multi-Genre Natural Language Inference (MNLI) benchmark [57]. A classification task typically used as a benchmark is the Quora Question Pairs (QQP). QQP consists of a binary classification task of whether two questions are paraphrases of each other.

In [8], weight magnitude pruning among was applied with the gradual pruning framework to an encoder-decoder Transformer for machine translation, achieving up to 70 % sparsity with a small decrease in performance. In [41] a first-order movement pruning method was applied to BERT. The method was evaluated for standard language understanding benchmarks at high sparsity levels of 90 % and 97 % sparsity and resulted in low to negligible performance reductions. The method achieved a 90 % sparsity with an accuracy reduction from 91.4 % to 90.5 % on the QQP task. In [58] a second-order gradual pruning method was applied to BERT surpassing the movement pruning approach in performance on the benchmark datasets. With the same amount of training epochs and the same sparsity level of 90 %, the method achieved an accuracy of 90.9 % on the QQP task. In [5] a post-training pruning framework utilizing a Fisher-information based mask variable optimization strategy was applied to BERT, achieving a QQP accuracy of 90.4 % without the need for any further training epochs.

Although direct comparison between the proposed method and the methods discussed here is challenging due to the different datasets and computing resources used, the general results point to the fact that genetic pruning in this case does not perform quite as well as many of the state-of-the-art pruning methods utilizing gradient information. Nevertheless, the results achieved by the genetic pruning can be seen as comparable with some of the results achieved by the discussed methods in classification tasks. As discussed earlier in this chapter, there also seems to be more room for improvement given more iterations and the current setup of the method. In addition, using a significantly higher genetic algorithm population size might be needed for optimal performance. Thus, a more comprehensive experiment with higher computational resources would be required for a

complete evaluation of the performance of the proposed method, which is beyond the scope of this thesis.

7.4.2 Limitations and Further Research Directions

The experimental case study presented in this chapter provides a good starting point for evaluating the performance of genetic pruning for Transformers. That being said, the presented experimental setup is quite narrow and exploratory in nature, thus there are definitely some limitations to it. The genetic pruning framework is quite computationally intensive and as the computational resources used to run the experiments are not comparable to most modern deep learning experiments, a comparison to other methods is not directly viable. The experiments were also restricted to text classification tasks, which represent only a single example of possible tasks that Transformer neural networks are used for. A broader experimental setup would also allow for a more detailed analysis of the effects of the choice of hyperparameters.

In addition to a more expansive benchmarking of the method, there are multiple other interesting research directions. Notably, the application of the genetic pruning method across diverse task domains, particularly those which do not allow for gradient-based learning, could be a fruitful avenue. A notable benefit of genetic algorithm based pruning approach is that it is highly flexible with regards to the model architecture and the performance metric being optimized. An essentially identical framework could be utilized for a wide variety of different neural network architectures, as the framework sets no restrictions on the structuring of the parameters being pruned.

There are many individual components of the genetic pruning method which could be examined in more detail. Specifically, the effect of the choice of crossover, mutation and selection implementations could be studied in order to find an optimal combination. Introducing novel genetic operations or approaches found in different variants of the basic genetic algorithm to the genetic pruning method provides opportunities for more research.

As mentioned in Chapter 6, it is quite straightforward to combine genetic pruning with other types of mask-based pruning methods which allows for locally searching around promising areas of the solution space found by the other method. Evaluating the effectiveness of this hybrid approach provides another interesting research direction. In conclusion, genetic algorithm based pruning methods provide many interesting research directions.

8. CONCLUSION

This thesis explored a novel way of utilizing genetic algorithms for pruning Transformers. The goal of the thesis was to introduce a new research direction for Transformer pruning and evaluate, whether genetic algorithm based pruning provides a viable framework for pruning Transformers. To achieve this, a broad theoretical background of the main components of the thesis was covered, including neural networks, Transformers, pruning and genetic algorithms. The proposed genetic pruning framework was introduced and an exploratory computational experiment was presented for text classification tasks with BERT, a Transformer based language model.

The conducted experiments provided promising preliminary evidence supporting the viability of genetic pruning as a framework for compressing Transformer models. The proposed pruning method achieved a substantial model compression with a small reduction in classification accuracy for the two benchmark datasets. Both the structured and unstructured version of the method found a better pruning solution than the baseline method with limited computational resources. Furthermore, the results indicate that there could be significant room for improvement within the framework by utilizing different hyperparameters and given more computing resources. The work indicates that genetic algorithm based pruning provides an interesting research direction and possibly a viable framework for pruning Transformers for text classification and likely other models and tasks.

However, while the results achieved by the method are promising, the exploratory nature of the experiments necessitates further, more comprehensive analyses to gain more insights into the performance of genetic pruning for Transformers. A broader experiment including varying model architectures, tasks, and datasets could robustly validate the effectiveness and versatility of the proposed framework. In addition, further research directions were identified in optimizing different components of the genetic pruning framework, introducing novel genetic operations to the algorithm and combining the genetic pruning framework with other model compression methods.

In conclusion, this thesis introduces genetic algorithm-based pruning as a promising novel framework for compressing Transformer models. The preliminary findings from the case study, while indicative of the method's potential, also highlight the need for a deeper and broader exploration to fully understand the benefits and limitations of genetic pruning.

REFERENCES

- [1] S. Dong, P. Wang, and K. Abbas, "A survey on deep learning and its applications", *Computer Science Review*, vol. 40, p. 100379, 2021, ISSN: 1574-0137. DOI: <https://doi.org/10.1016/j.cosrev.2021.100379>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1574013721000198>.
- [2] J. e. a. Wei, "Emergent abilities of large language models", *Transactions on Machine Learning Research*, 2022, Survey Certification, ISSN: 2835-8856. [Online]. Available: <https://openreview.net/forum?id=yzkSU5zdwD>.
- [3] e. a. Xin Zhao Wayne, *A survey of large language models*, 2023. arXiv: 2303.18223 [cs.CL].
- [4] S. e. a. Bubeck, *Sparks of artificial general intelligence: Early experiments with gpt-4*, 2023. arXiv: 2303.12712 [cs.CL].
- [5] W. e. a. Kwon, "A fast post-training pruning framework for transformers", in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35, Curran Associates, Inc., 2022, pp. 24101–24116. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/file/987bed997ab668f91c822a09bce3ea12-Paper-Conference.pdf.
- [6] Z. Li, H. Li, and L. Meng, "Model compression for deep neural networks: A survey", *Computers*, vol. 12, no. 3, 2023, ISSN: 2073-431X. DOI: 10.3390/computers12030060. [Online]. Available: <https://www.mdpi.com/2073-431X/12/3/60>.
- [7] T. Liang, J. Glossner, L. Wang, S. Shi, and X. Zhang, "Pruning and quantization for deep neural network acceleration: A survey", *Neurocomputing*, vol. 461, pp. 370–403, 2021, ISSN: 0925-2312. DOI: <https://doi.org/10.1016/j.neucom.2021.07.045>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0925231221010894>.
- [8] T. Gale, E. Elsen, and S. Hooker, *The state of sparsity in deep neural networks*, 2019. DOI: 10.48550/ARXIV.1902.09574. [Online]. Available: <https://arxiv.org/abs/1902.09574>.
- [9] Y. Liu, Y. Sun, B. Xue, M. Zhang, G. G. Yen, and K. C. Tan, "A survey on evolutionary neural architecture search", *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 2, pp. 550–570, 2023. DOI: 10.1109/TNNLS.2021.3100554.
- [10] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016, <http://www.deeplearningbook.org>.

- [11] N. Buduma and N. Locascio, *Fundamentals of Deep Learning: Designing Next-Generation Machine Intelligence Algorithms*, 1st. O'Reilly Media, Inc., 2017, ISBN: 1491925612.
- [12] F. Rosenblatt, "The perceptron: A probabilistic model for information storage and organization in the brain", *Psychological Review*, vol. 65(6), pp. 386–408, 1958.
- [13] B. e. a. Xu, *Empirical evaluation of rectified activations in convolutional network*, 2015. arXiv: 1505.00853 [cs.LG].
- [14] D. Hendrycks and K. Gimpel, *Gaussian error linear units (gelus)*, 2016. DOI: 10.48550/ARXIV.1606.08415. [Online]. Available: <https://arxiv.org/abs/1606.08415>.
- [15] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning", *Nature*, vol. 521, no. 7553, pp. 436–444, May 2015, ISSN: 1476-4687. DOI: 10.1038/nature14539. [Online]. Available: <https://doi.org/10.1038/nature14539>.
- [16] J. Duchi, E. Hazan, and Y. Singer, "Adaptive subgradient methods for online learning and stochastic optimization", *J. Mach. Learn. Res.*, vol. 12, no. null, pp. 2121–2159, Jul. 2011, ISSN: 1532-4435.
- [17] M. D. Zeiler, *Adadelata: An adaptive learning rate method*, 2012. DOI: 10.48550/ARXIV.1212.5701. [Online]. Available: <https://arxiv.org/abs/1212.5701>.
- [18] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting", *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014. [Online]. Available: <http://jmlr.org/papers/v15/srivastava14a.html>.
- [19] K. Cho, B. van Merriënboer, D. Bahdanau, and Y. Bengio, "On the properties of neural machine translation: Encoder-decoder approaches", *CoRR*, vol. abs/1409.1259, 2014. arXiv: 1409.1259. [Online]. Available: <http://arxiv.org/abs/1409.1259>.
- [20] K. He, X. Zhang, S. Ren, and J. Sun, *Deep residual learning for image recognition*, 2015. arXiv: 1512.03385 [cs.CV].
- [21] S. Hochreiter and J. Schmidhuber, "Long short-term memory", *Neural computation*, vol. 9, pp. 1735–80, Dec. 1997. DOI: 10.1162/neco.1997.9.8.1735.
- [22] T. N. Kipf and M. Welling, *Semi-supervised classification with graph convolutional networks*, 2017. arXiv: 1609.02907 [cs.LG].
- [23] B. Hanin, "Universal function approximation by deep neural nets with bounded width and ReLU activations", *Mathematics*, vol. 7, no. 10, p. 992, Oct. 2019. DOI: 10.3390/math7100992. [Online]. Available: <https://doi.org/10.3390/math7100992>.
- [24] X. Wang, K. Wen, Z. Zhang, L. Hou, Z. Liu, and J. Li, *Finding skill neurons in pre-trained transformer-based language models*, 2022. DOI: 10.48550/ARXIV.2211.07349. [Online]. Available: <https://arxiv.org/abs/2211.07349>.

- [25] T. Chen, J. Frankle, S. Chang, *et al.*, *The lottery ticket hypothesis for pre-trained bert networks*, 2020. DOI: 10.48550/ARXIV.2007.12223. [Online]. Available: <https://arxiv.org/abs/2007.12223>.
- [26] V. Kozhevnikov and E. Pankratova, "Research of the text data vectorization and classification algorithms of machine learning", *Theoretical & Applied Science*, vol. 85, Jun. 2020. DOI: 10.15863/TAS.2020.05.85.106.
- [27] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, *Bert: Pre-training of deep bidirectional transformers for language understanding*, 2018. DOI: 10.48550/ARXIV.1810.04805. [Online]. Available: <https://arxiv.org/abs/1810.04805>.
- [28] D. Bahdanau, K. Cho, and Y. Bengio, *Neural machine translation by jointly learning to align and translate*, 2016. arXiv: 1409.0473 [cs.CL].
- [29] Y. Wu, M. Schuster, Z. Chen, *et al.*, *Google's neural machine translation system: Bridging the gap between human and machine translation*, 2016. arXiv: 1609.08144 [cs.CL].
- [30] A. Vaswani, N. Shazeer, N. Parmar, *et al.*, *Attention is all you need*, 2017. DOI: 10.48550/ARXIV.1706.03762. [Online]. Available: <https://arxiv.org/abs/1706.03762>.
- [31] G. Brauwers and F. Frasincar, "A general survey on attention mechanisms in deep learning", *IEEE Transactions on Knowledge and Data Engineering*, pp. 1–1, 2021. DOI: 10.1109/tkde.2021.3126456. [Online]. Available: <https://doi.org/10.1109/tkde.2021.3126456>.
- [32] M.-T. Luong, H. Pham, and C. D. Manning, *Effective approaches to attention-based neural machine translation*, 2015. DOI: 10.48550/ARXIV.1508.04025. [Online]. Available: <https://arxiv.org/abs/1508.04025>.
- [33] J. L. Ba, J. R. Kiros, and G. E. Hinton, *Layer normalization*, 2016. DOI: 10.48550/ARXIV.1607.06450. [Online]. Available: <https://arxiv.org/abs/1607.06450>.
- [34] F. D. Keles, P. M. Wijewardena, and C. Hegde, *On the computational complexity of self-attention*, 2022. arXiv: 2209.04881 [cs.LG].
- [35] Y. Liu, M. Ott, N. Goyal, *et al.*, *Roberta: A robustly optimized bert pretraining approach*, 2019. arXiv: 1907.11692 [cs.CL].
- [36] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, *Distilbert, a distilled version of bert: Smaller, faster, cheaper and lighter*, 2020. arXiv: 1910.01108 [cs.CL].
- [37] C. Sun, X. Qiu, Y. Xu, and X. Huang, "How to fine-tune bert for text classification?", in *Chinese Computational Linguistics*, M. Sun, X. Huang, H. Ji, Z. Liu, and Y. Liu, Eds., Cham: Springer International Publishing, 2019, pp. 194–206, ISBN: 978-3-030-32381-3.
- [38] D. Blalock, J. J. G. Ortiz, J. Frankle, and J. Gutttag, *What is the state of neural network pruning?*, 2020. DOI: 10.48550/ARXIV.2003.03033. [Online]. Available: <https://arxiv.org/abs/2003.03033>.

- [39] L. A. BRESLOW and D. W. AHA, "Simplifying decision trees: A survey", *The Knowledge Engineering Review*, vol. 12, no. 01, pp. 1–40, 1997. DOI: 10.1017/S0269888997000015.
- [40] P. Molchanov, S. Tyree, T. Karras, T. Aila, and J. Kautz, "Pruning convolutional neural networks for resource efficient transfer learning", *CoRR*, vol. abs/1611.06440, 2016. arXiv: 1611.06440. [Online]. Available: <http://arxiv.org/abs/1611.06440>.
- [41] V. Sanh, T. Wolf, and A. M. Rush, *Movement pruning: Adaptive sparsity by fine-tuning*, 2020. DOI: 10.48550/ARXIV.2005.07683. [Online]. Available: <https://arxiv.org/abs/2005.07683>.
- [42] Y. LeCun, J. Denker, and S. Solla, "Optimal brain damage", in *Advances in Neural Information Processing Systems*, D. Touretzky, Ed., vol. 2, Morgan-Kaufmann, 1989. [Online]. Available: <https://proceedings.neurips.cc/paper/1989/file/6c9882bbac1c7093bd25041881277658-Paper.pdf>.
- [43] S. Han, J. Pool, J. Tran, and W. J. Dally, *Learning both weights and connections for efficient neural networks*, 2015. DOI: 10.48550/ARXIV.1506.02626. [Online]. Available: <https://arxiv.org/abs/1506.02626>.
- [44] M. Zhu and S. Gupta, *To prune, or not to prune: Exploring the efficacy of pruning for model compression*, 2017. DOI: 10.48550/ARXIV.1710.01878. [Online]. Available: <https://arxiv.org/abs/1710.01878>.
- [45] J. Holland, *Adaptation in natural and artificial systems: An introductory analysis with applications to biology, control, and artificial intelligence*. U Michigan Press, 1975.
- [46] M. Gen, R. Cheng, and L. Lin, *Network Models and Optimization, Multiobjective Genetic Algorithm Approach*, ser. Decision Engineering. Springer London, 2008, ISBN: 978-1-84800-181-7.
- [47] H. M. Pandey, A. Shukla, A. Chaudhary, and D. Mehrotra, "Evaluation of genetic algorithm's selection methods", eng, in *Information Systems Design and Intelligent Applications*, ser. Advances in Intelligent Systems and Computing, New Delhi: Springer India, 2016, pp. 731–738, ISBN: 8132227506.
- [48] O. Kramer, *Genetic Algorithm Essentials*, eng, 1st ed. 2017., ser. Studies in Computational Intelligence, 679. Cham: Springer International Publishing, 2017, ISBN: 3-319-52156-X.
- [49] "A comparison of selection schemes used in evolutionary algorithms", eng, *Evolutionary computation*, vol. 4, no. 4, pp. 361–394, 1996, ISSN: 1063-6560.
- [50] K. O. Stanley and R. Miikkulainen, "Evolving neural networks through augmenting topologies", *Evolutionary Computation*, vol. 10, no. 2, pp. 99–127, 2002. DOI: 10.1162/106365602320169811.
- [51] C. Yang, Z. An, C. Li, B. Diao, and Y. Xu, *Multi-objective pruning for cnns using genetic algorithm*, 2019. DOI: 10.48550/ARXIV.1906.00399. [Online]. Available: <https://arxiv.org/abs/1906.00399>.

- [52] T. Wolf, L. Debut, V. Sanh, *et al.*, *Huggingface's transformers: State-of-the-art natural language processing*, 2019. DOI: 10.48550/ARXIV.1910.03771. [Online]. Available: <https://arxiv.org/abs/1910.03771>.
- [53] A. Paszke, S. Gross, F. Massa, *et al.*, "Pytorch: An imperative style, high-performance deep learning library", in *Advances in Neural Information Processing Systems 32*, Curran Associates, Inc., 2019, pp. 8024–8035. [Online]. Available: <http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf>.
- [54] X. Zhang, J. Zhao, and Y. LeCun, *Character-level convolutional networks for text classification*, 2016. arXiv: 1509.01626 [cs.LG].
- [55] A. Maas, R. Daly, P. Pham, D. Huang, A. Ng, and C. Potts, "Learning word vectors for sentiment analysis", Jan. 2011, pp. 142–150.
- [56] E. Gordon-Rodriguez, G. Loaiza-Ganem, G. Pleiss, and J. P. Cunningham, *Uses and abuses of the cross-entropy loss: Case studies in modern deep learning*, 2020. arXiv: 2011.05231 [stat.ML].
- [57] A. Williams, N. Nangia, and S. Bowman, "A broad-coverage challenge corpus for sentence understanding through inference", in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, New Orleans, Louisiana: Association for Computational Linguistics, 2018, pp. 1112–1122. [Online]. Available: <http://aclweb.org/anthology/N18-1101>.
- [58] E. Kurtic, D. Campos, T. Nguyen, *et al.*, *The optimal bert surgeon: Scalable and accurate second-order pruning for large language models*, 2022. DOI: 10.48550/ARXIV.2203.07259. [Online]. Available: <https://arxiv.org/abs/2203.07259>.